

Day0399 -- IPC127 Embedded Industrial PC and Its Role as an IoT Gateway

เจ้าของงาน ID:B6728960 Name:มงคล น้อยบาท

ผู้ทดลอง ID: B6729615 Name:ปภักร สมสุข

- Download >>
https://drive.google.com/drive/folders/13u1PiPoJfqIU2HCr_gSHpj5Sb7j14KyV?usp=drive_link
- ทำการทดลอง เก็บข้อมูล ลองคนต่อกลุ่ม แล้วแยกกันเติมรายละเอียดรายงานให้เรียบร้อย
- กำหนดชื่อไฟล์ตามรูปแบบ "B3601234 - Wichai Srisuruk – Day0399 IPC127 Embedded PC.pdf"
- ส่งแบบ pdf รอบโครงสร้าง ก่อน 20260816-1700 ที่
<https://forms.gle/o9yNMLLyza4NdoFd9>
- ส่งแบบ pdf รอบสมบูรณ์ ก่อน 20260830-0600 ที่
<https://forms.gle/o9yNMLLyza4NdoFd9>

Read More

- <https://www.mpen.co.th/17256294/siemens-simatic-ipc127>
- <https://circuitdigest.com/microcontroller-projects/controlling-leds-using-hand-gestures-with-esp32-and-python>
- <https://docs.aic-eec.com/biil/iot-connectivity-and-data-analytics-via-node-red/node-red-installation>
- <https://www.sysadmin.in.th/install-node-red-on-windows/>
- <https://grafana.com/grafana/download>
- <https://grafana.com/docs/grafana/latest/setup-grafana/installation/windows/>
- <https://grafana.com/grafana/plugins/grafana-mqtt-datasource/>
- <https://grafana.com/grafana/plugins/speakyourcode-button-panel/>
- <https://www.influxdata.com/>
- <https://grafana.com/docs/grafana/latest/getting-started/get-started-grafana-influxdb/>
- <https://www.artronshop.co.th/article/162/mqtt-broker-with-emqx-and-raspberry-pi-5>

- <https://www.emqx.com/en/blog/install-mqtt-broker-on-windows>
- <https://cedalo.com/blog/how-to-install-mosquitto-mqtt-broker-on-windows/>

Mission 1/7 -- Getting Started

1. ติดตั้ง Win 10 IOT บน IPC127 >> กำหนด login=cpe28sut, password=adminZ01

ภาพการทำงาน 1/4

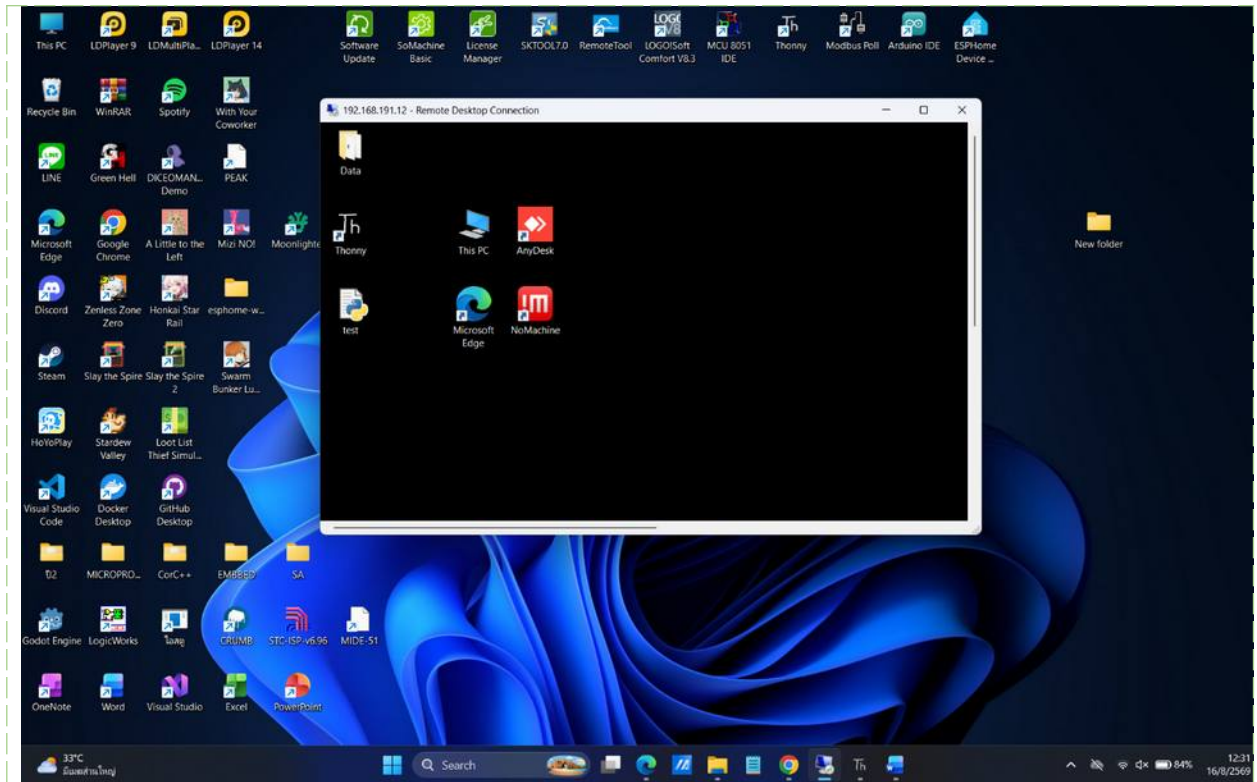
ภาพการทำงาน 2/4

ภาพการทำงาน 3/4

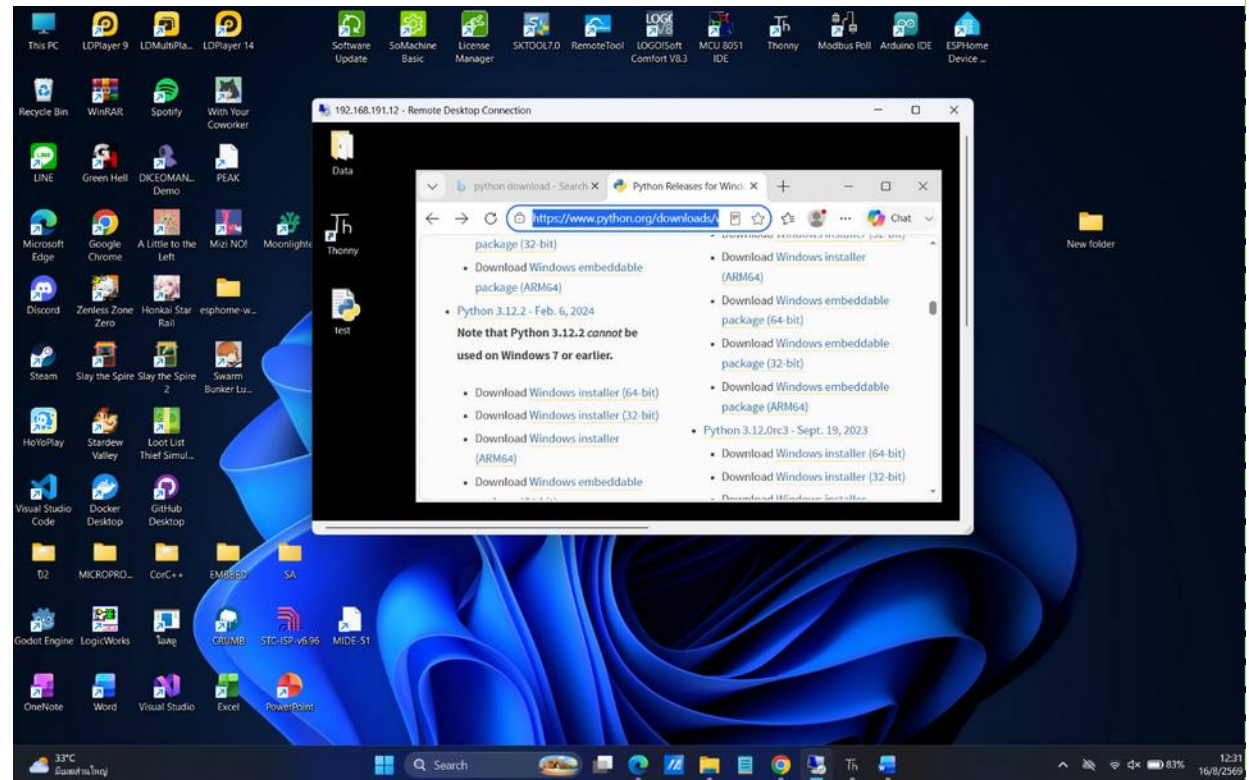
ภาพการทำงาน 4/4

2. เปิดใช้งาน Remote Desktop

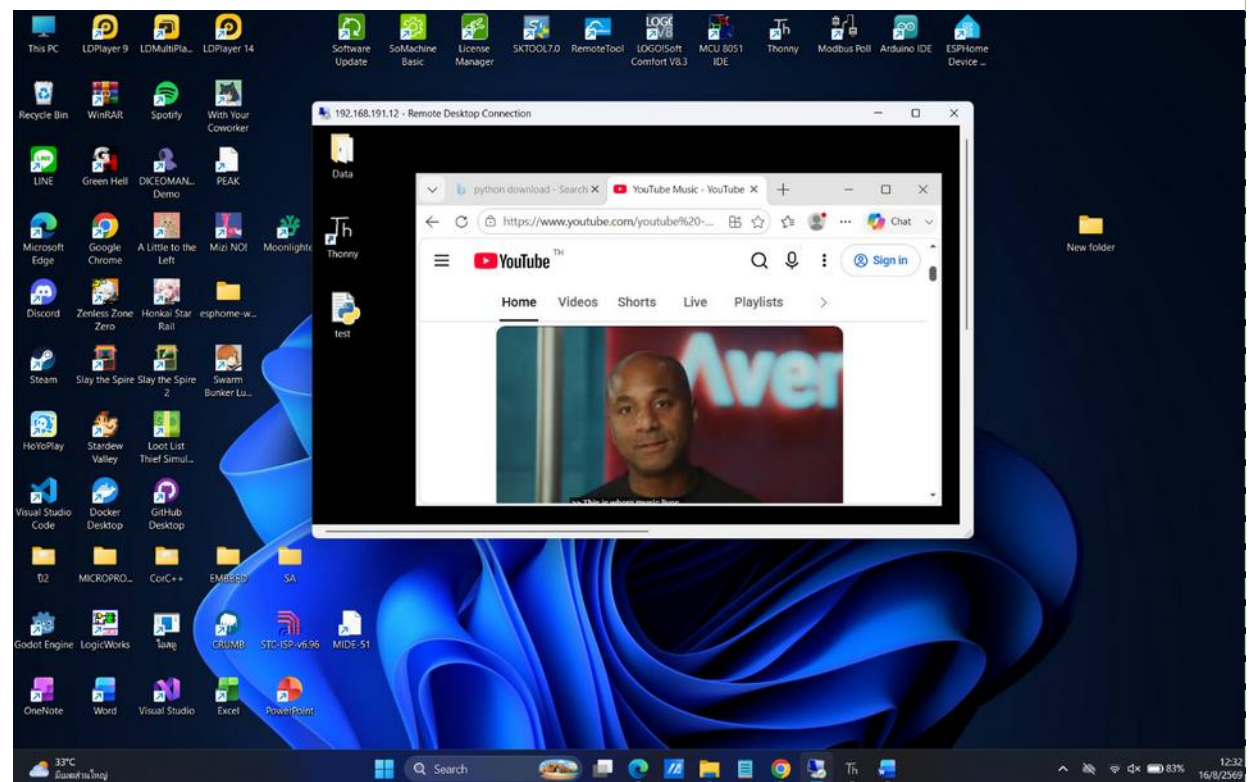
ภาพการทำงาน 1/4



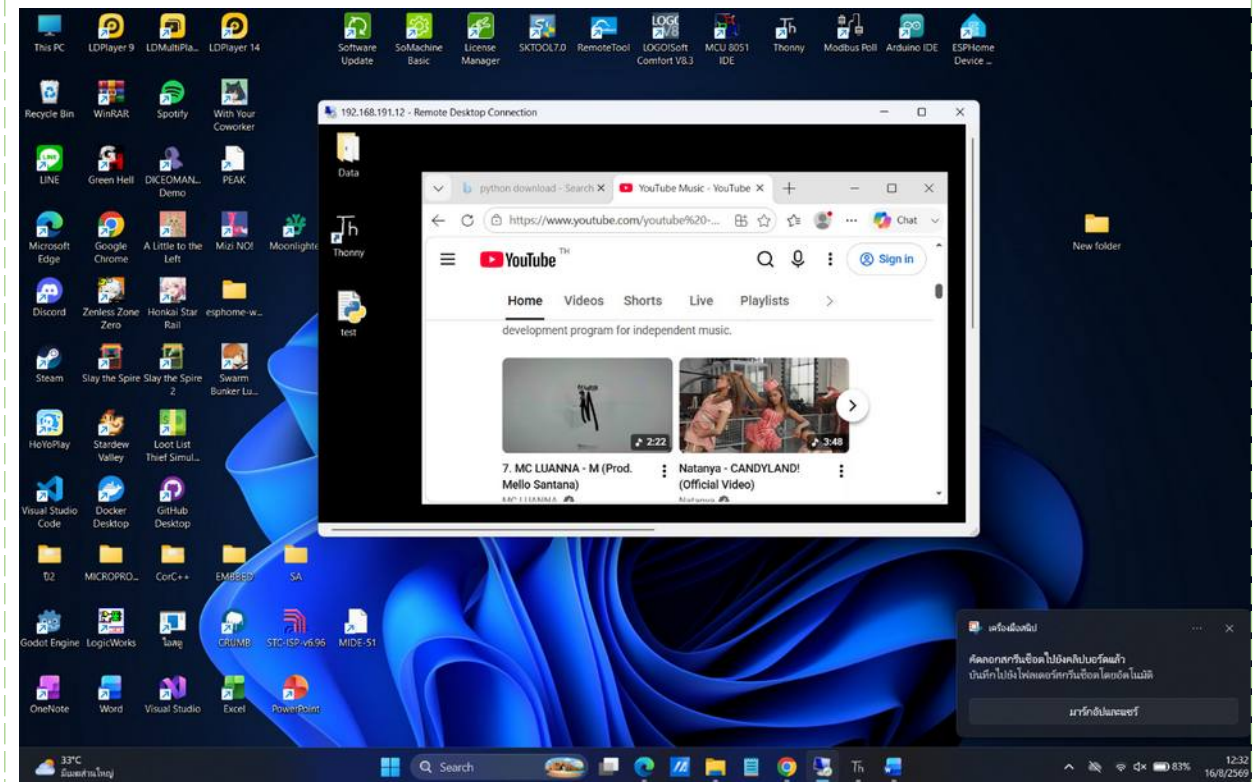
ภาพการทำงาน 2/4



ภาพการทำงาน 3/4



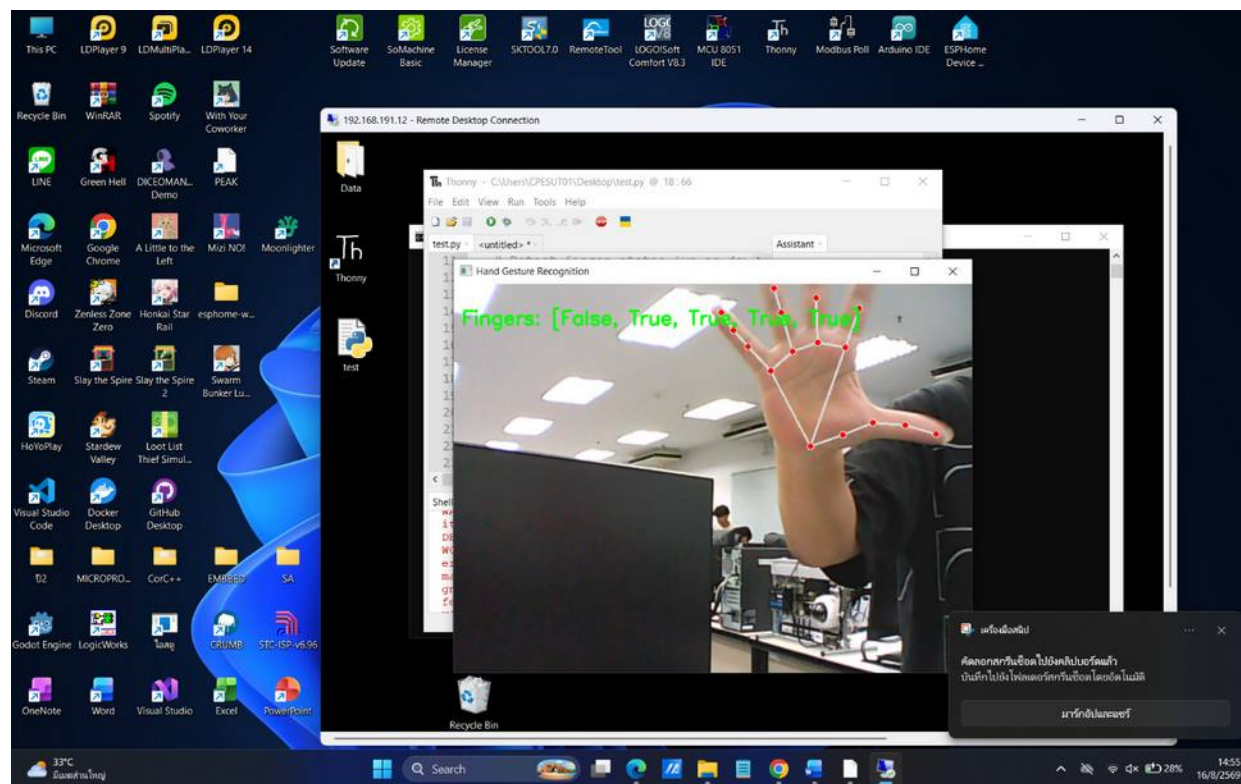
ภาพการทำงาน 4/4



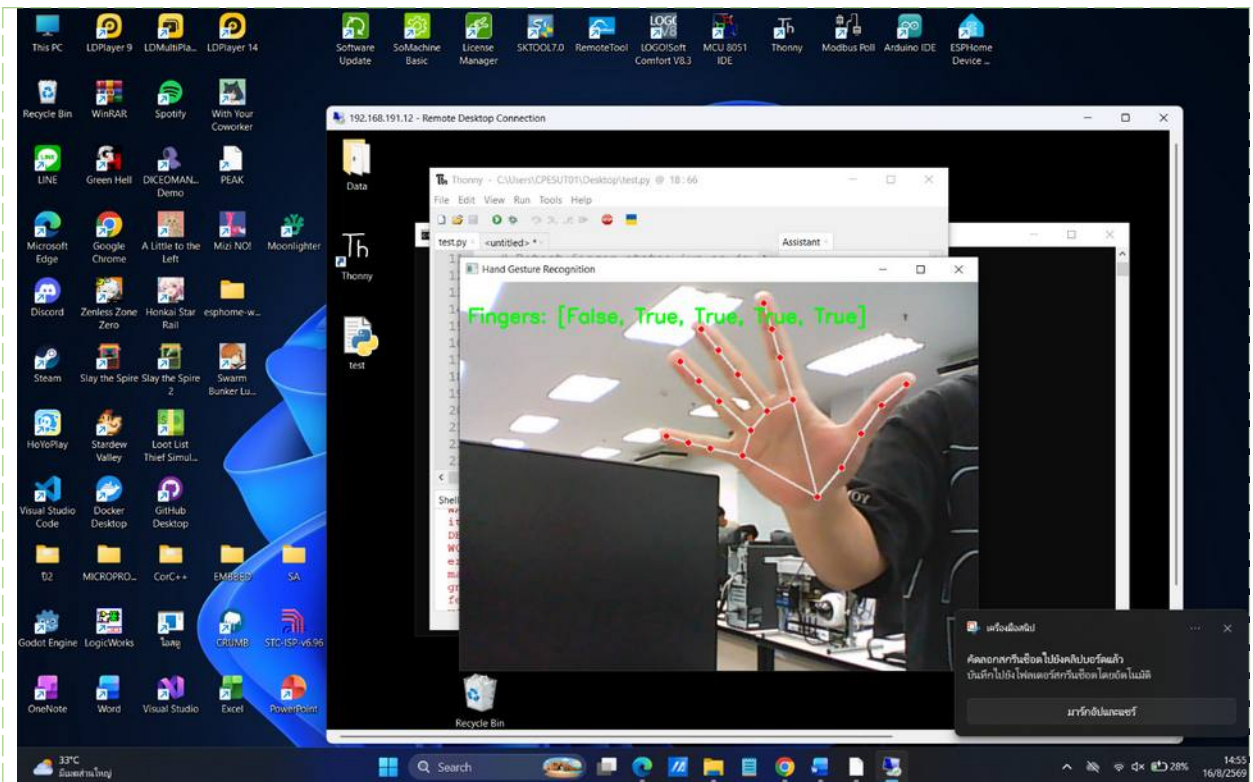
Mission 2/7 – Image Processing

1. ติดตั้ง Thony Python และทดสอบการทำงาน
2. Add Library
 - o Install OpenCV pip install opencv-python
 - o Install mediapipe pip install mediapipe
 - o Install requests pip install requests
3. ทดสอบ mediapipe

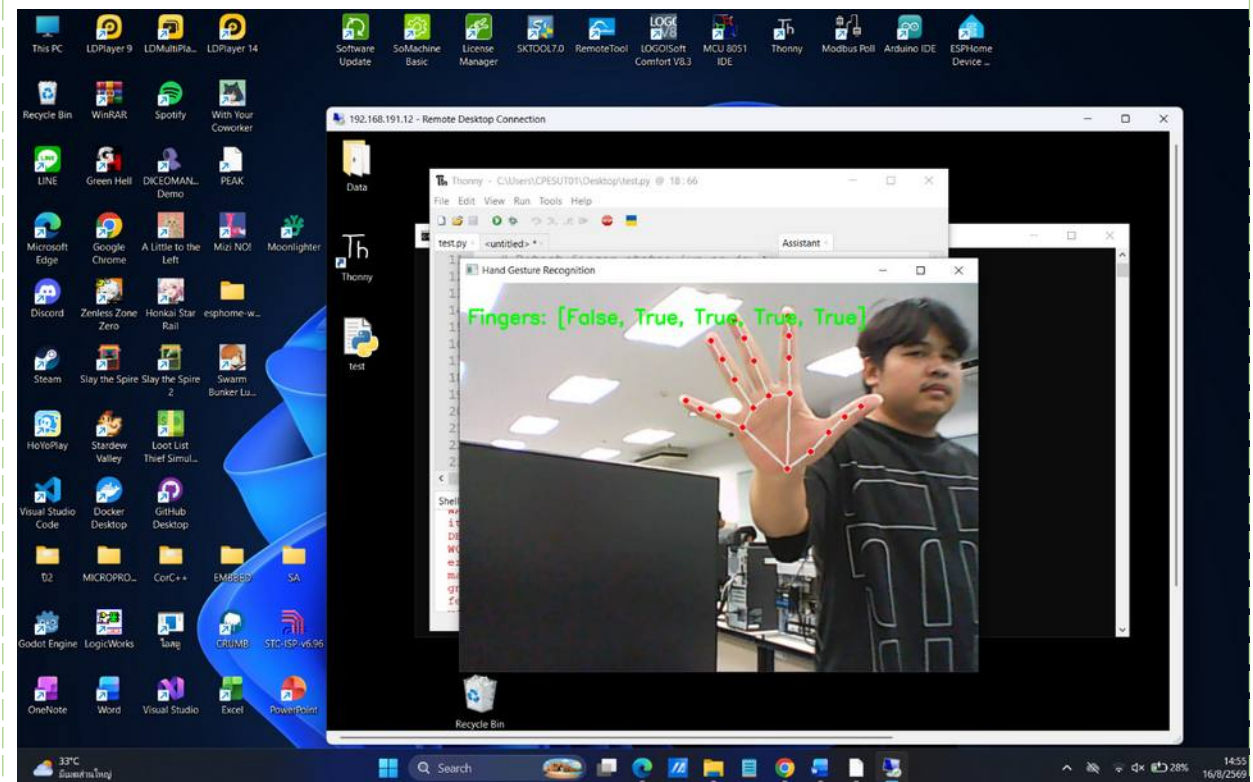
ภาพการทำงาน 1/3



ภาพการทำงาน 2/3

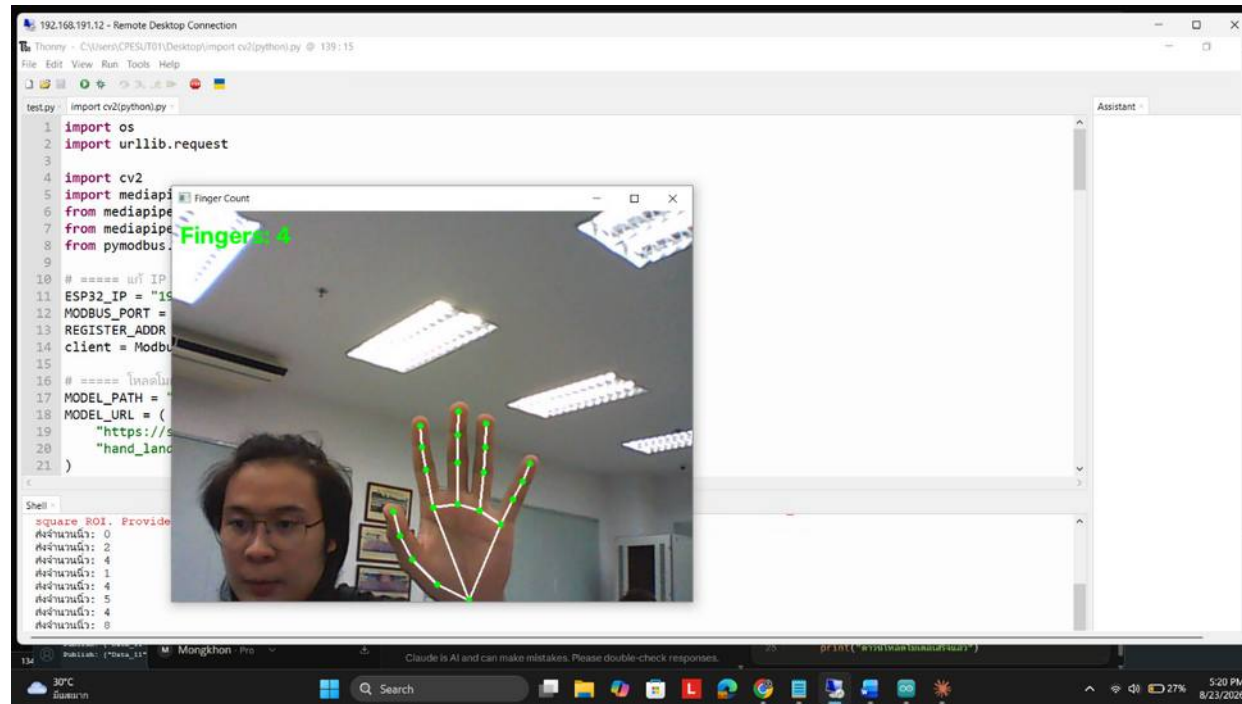


ภาพการทำงาน 3/3

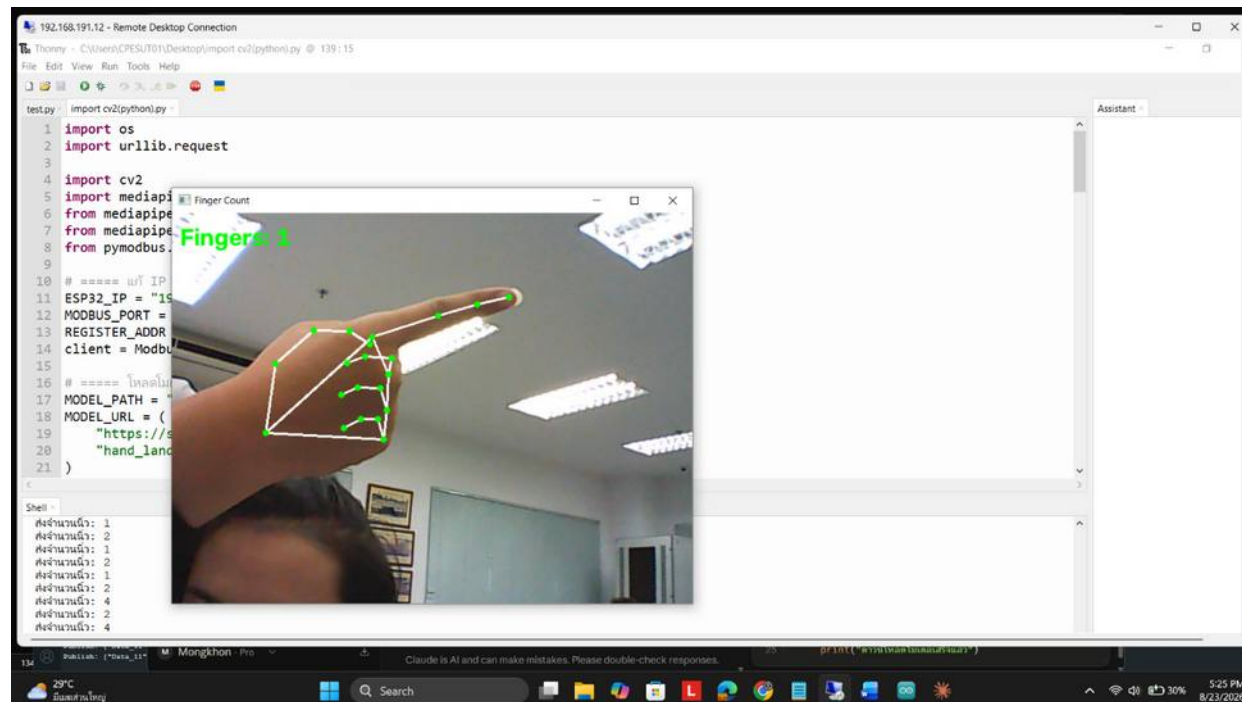


4. ทดสอบ mediapipe ส่งค่าไปแสดงที่ 5 LED

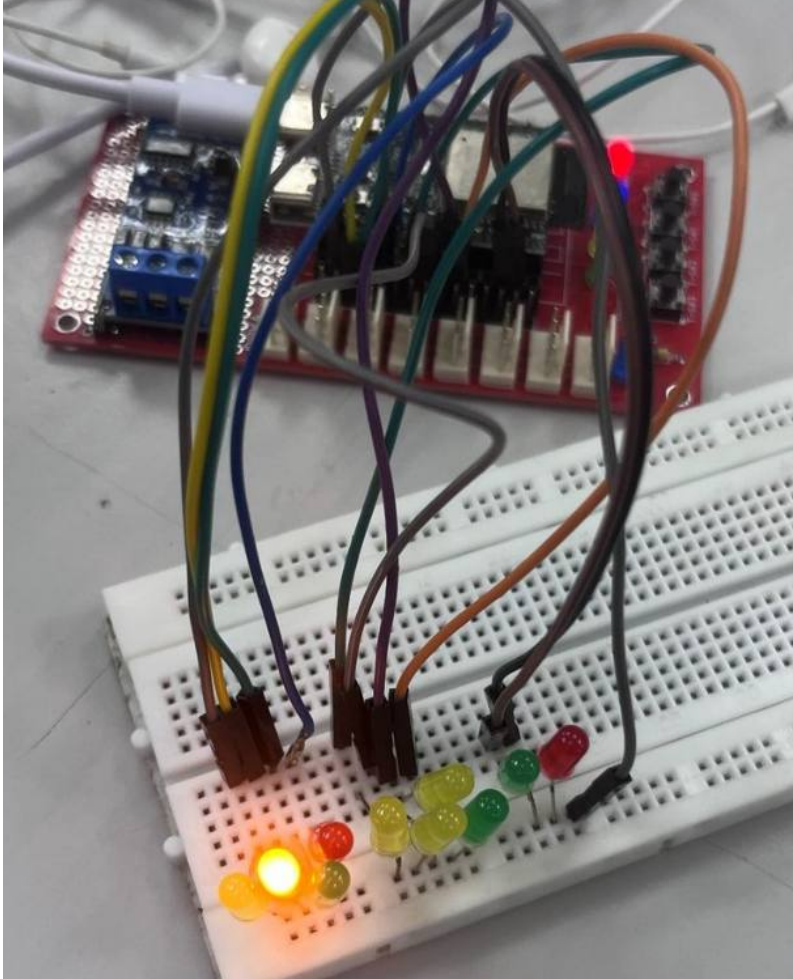
ภาพการทำงาน 1/3



ภาพการทำงาน 2/3

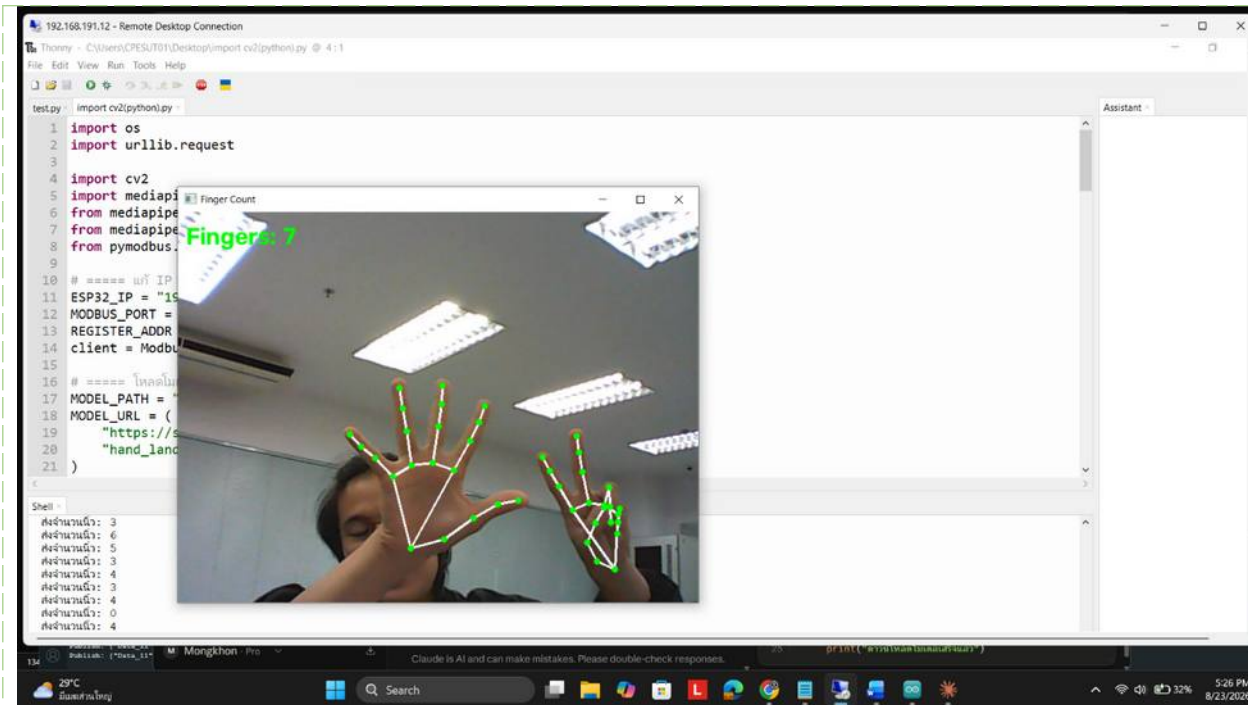


ภาพการทำงาน 3/3

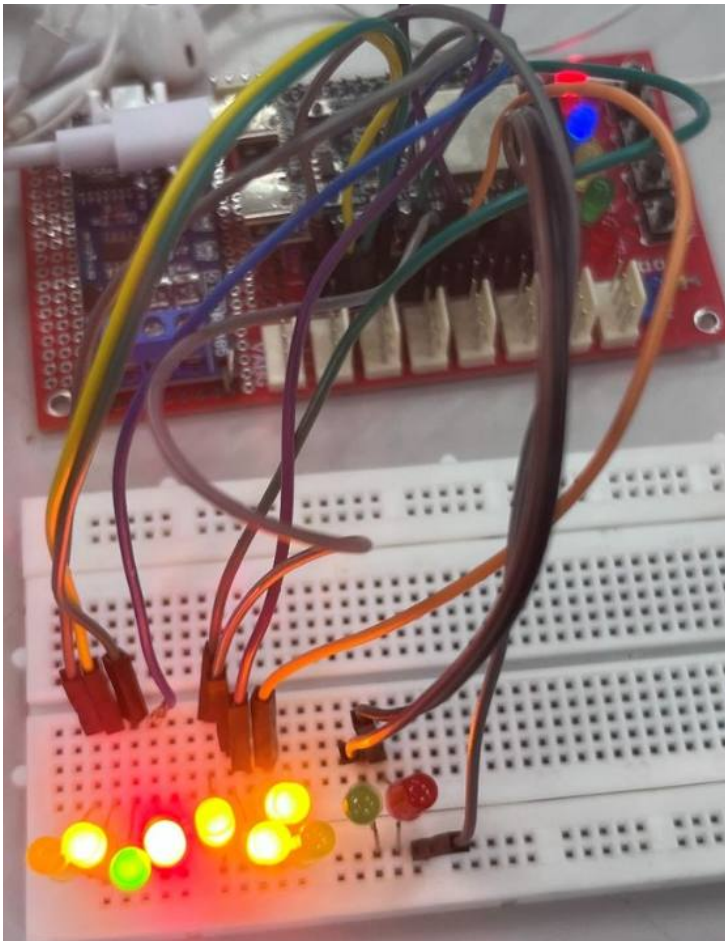


5. ทดสอบ mediapipe ส่งค่าไปแสดงที่ 10 LED

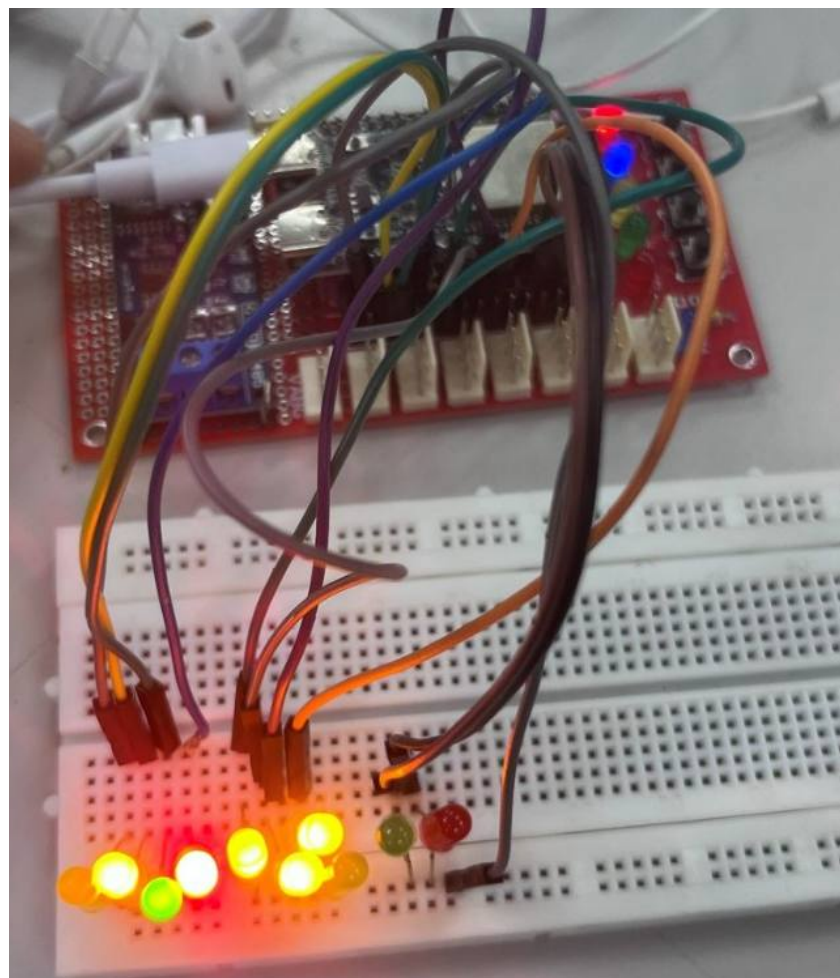
ภาพการทำงาน 1/3



ภาพการทำงาน 2/3



ภาพการทำงาน 3/3



Mission 3/7 – Embedded System based on Windows Device

1. ปรับแต่งให้ โปรแกรมทำงานทันทีเมื่อเปิดเครื่อง IPC217

ภาพการทำงาน 1/4

ภาพการทำงาน 2/4

ภาพการทำงาน 3/4

ภาพการทำงาน 4/4

Mission 4/7: Software Install

1. NodeRED

ขั้นตอนติดตั้ง 1

เปิด PowerShell แบบ Run as Administrator

```
npm install -g --unsafe-perm node-red
```

ขั้นตอนติดตั้ง 2

เช็กว่าติดตั้งสำเร็จ:

```
node-red --version
```

ขั้นตอนติดตั้ง 3

Step 3 — รัน Node-RED ครั้งแรก

ใน PowerShell:

```
node-red
```

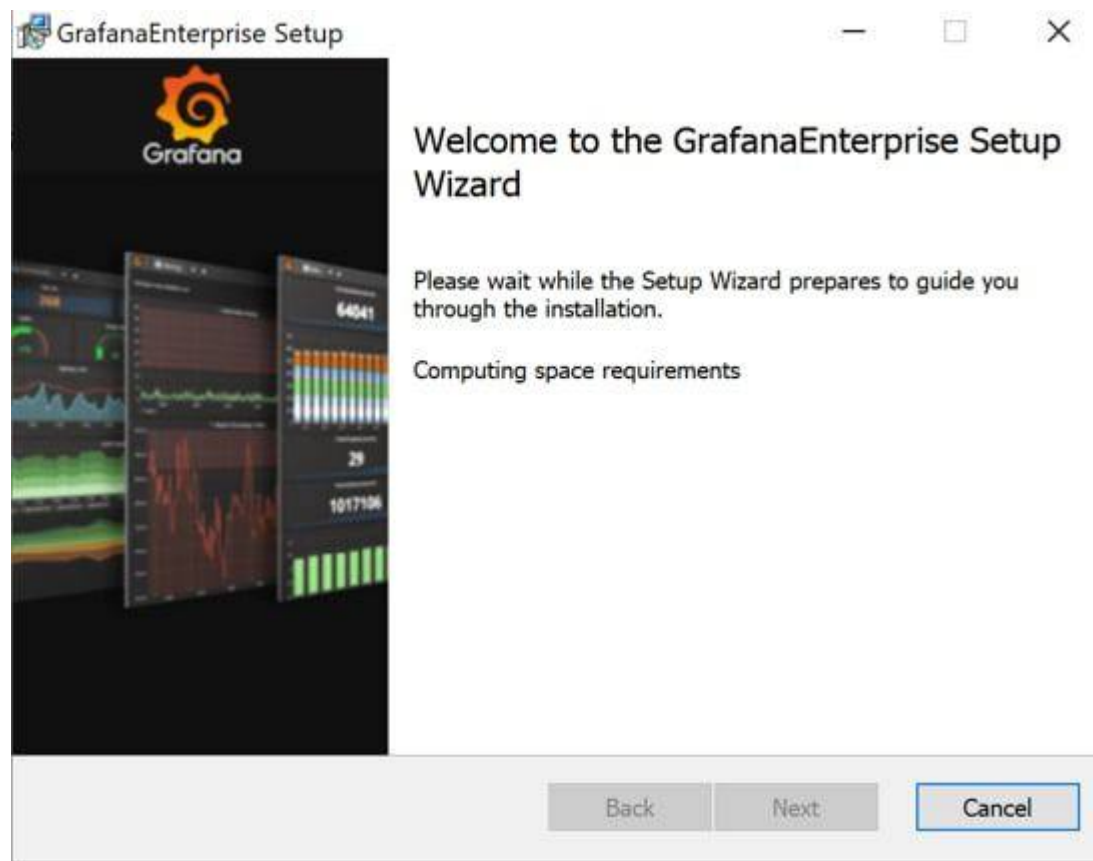
2. Grafana

ขั้นตอนติดตั้ง 1

Step 1 — ดาวน์โหลด installer

1. ไปที่ <https://grafana.com/grafana/download?platform=windows>

ขั้นตอนติดตั้ง 2



ขั้นตอนติดตั้ง 3



3. InfluxDB

ขั้นตอนติดตั้ง 1



ขั้นตอนติดตั้ง 2

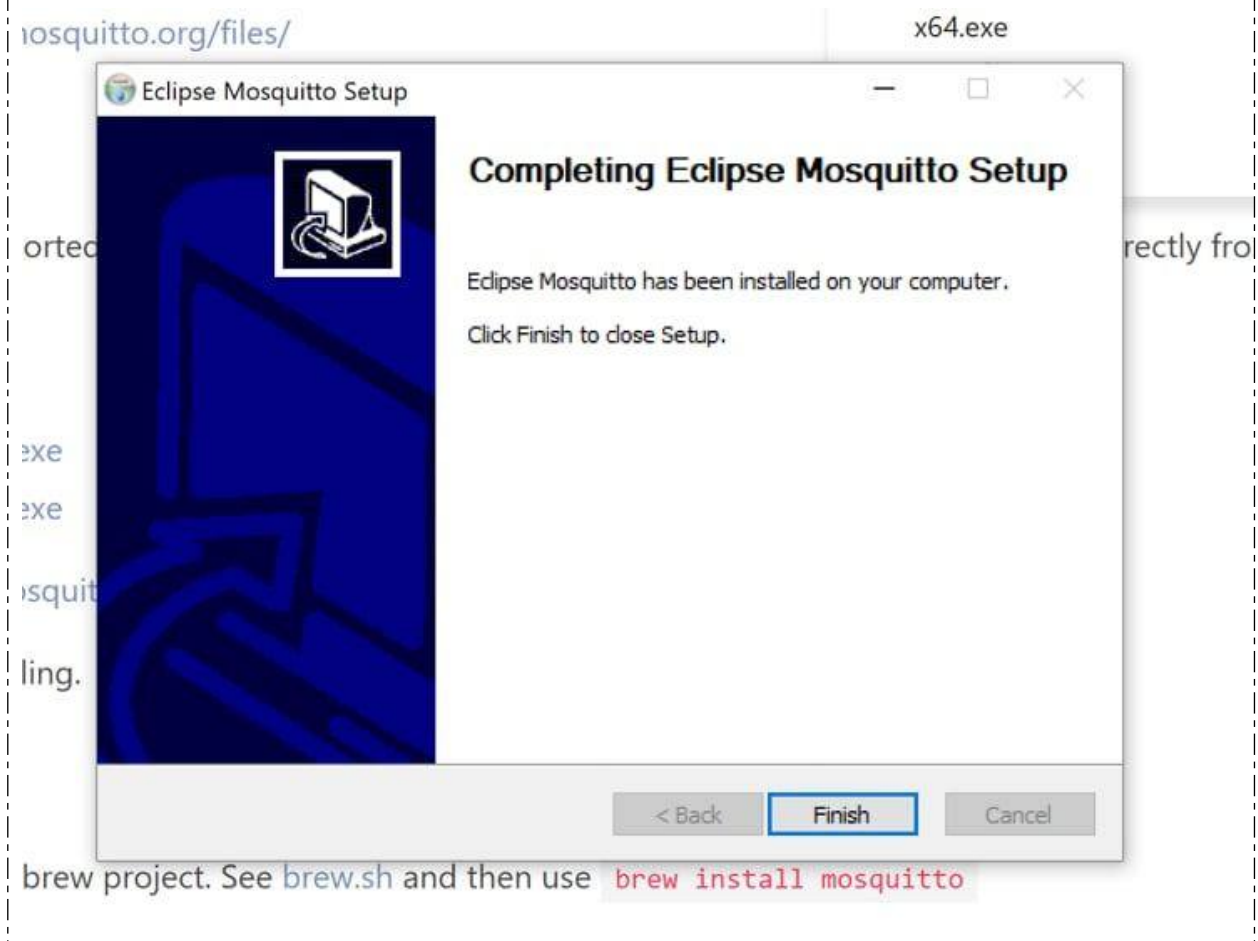
```
curl -O https://www.influxdata.com/d/install_influxdb3.sh &&  
sh install_influxdb3.sh
```

ขั้นตอนติดตั้ง 3

```
C:\InfluxDB>influxd version  
InfluxDB v2.7.11 (git: fbf5d4ab5e) build_date: 2024-12-02T17:48:13Z
```

4. MQTT Broker

ขั้นตอนติดตั้ง 1



brew project. See [brew.sh](#) and then use `brew install mosquitto`

ขั้นตอนติดตั้ง 2

```
Command Prompt

C:\Users\CPESUT01>cd C:\Program Files\Mosquitto

C:\Program Files\Mosquitto>mosquitto -h
mosquitto version 2.1.2

mosquitto is an MQTT v5.0/v3.1.1/v3.1 broker.

Usage: mosquitto [-c config_file] [-d] [-h] [-p port] [-v]
               [--tls-keylog file]

-c : specify the broker config file.
-d : put the broker into the background after starting.
-h : display this help.
-p : start the broker listening on the specified port.
    Not recommended in conjunction with the -c option.
-q : quiet mode - disable all logging types. This overrides
    any logging options given in the config file, and -v.
-v : verbose mode - enable all logging types. This overrides
    any logging options given in the config file.
--test-config : test config file and exit
--tls-keylog : log TLS connection information to a file, to allow
    debugging with e.g. Wireshark. Do not use on a production
    server.

See https://mosquitto.org/ for more information.

1787121132: mosquitto version 2.1.2 terminating

C:\Program Files\Mosquitto>
```

Mission 5/7: Data to Dashboard

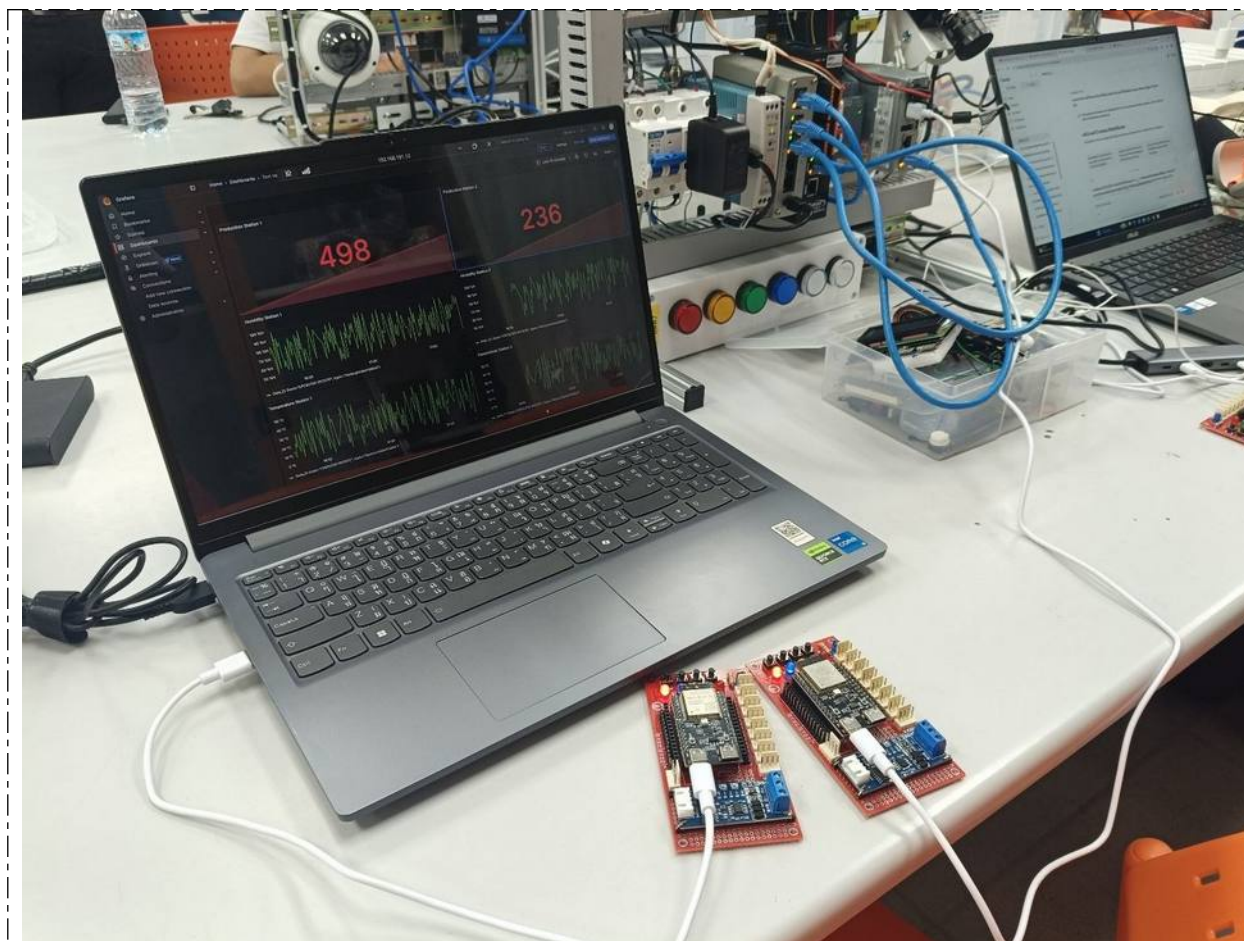
1. ภาพผังการทำงานโดยรวม



2. ใช้ ESP32 จำนวน สองตัว ส่งข้อมูลไปเก็บที่ InfluxDB บน IPC127

- สถานี_1 ESP1 Data_11: อุณหภูมิ -> Random [0.00-49.99]
- สถานี_1 ESP1 Data_12: ความชื้น -> Random [50.00-99.99]
- สถานี_1 ESP1 Data_13: จำนวนที่ผลิต -> Step Up [0-999999]
- สถานี_2 ESP2 Data_21: อุณหภูมิ -> Random [0.00-49.99]
- สถานี_2 ESP2 Data_22: ความชื้น -> Random [50.00-99.99]
- สถานี_2 ESP2 Data_23: จำนวนที่ผลิต -> Step Up [0-999999]

ภาพถ่ายวงจร ESP32 ขณะทดสอบ



ESP32 Code Picture

```

File Edit Sketch Tools Help
ESP32C6 Dev Module

Embedded_West06.ino
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 // ===== Wi-Fi =====
5 const char* ssid = "RUT956_T07";
6 const char* password = "12345667";
7
8 // ===== MQTT Broker =====
9 const char* mqtt_server = "192.168.1.165";
10 const int mqtt_port = 1883;
11
12 // Topic ของสถานี 1
13 const char* mqtt_topic = "factory/station2/data";
14
15 WiFiClient espClient;
16 PubSubClient client(espClient);
17
18 unsigned long lastMsg = 0;
19 unsigned long production = 0;
20
21 void setup_wifi() {
22   delay(10);
23   Serial.println();
24   Serial.print("Connecting to WiFi: ");
25   Serial.println(ssid);
26
27   WiFi.begin(ssid, password);
28
29   while (WiFi.status() != WL_CONNECTED) {
30     delay(500);
31     Serial.print(".");
32   }
33
34   Serial.println();
35   Serial.println("WiFi connected");
36   Serial.print("ESP32 IP: ");
37   Serial.println(WiFi.localIP());
38 }
39
40 void reconnectMQTT() {
41   while (!client.connected()) {
42     Serial.print("Connecting to MQTT...");
43
44     String clientId = "ESP32C6-Station1-";
45     clientId += String(random(0xffff), HEX);
46
47     if (client.connect(clientId.c_str())) {
48       Serial.println("connected");
49     } else {
50       Serial.print("failed, rc=");
51       Serial.print(client.state());
52       Serial.println(" retry in 5 seconds");
53       delay(5000);
54     }
55   }
56 }
57
58 void setup() {
59   Serial.begin(115200);
60   randomSeed(micros());
61   setup_wifi();
62   client.setServer(mqtt_server, mqtt_port);
63 }
64
65 void loop() {
66   if (!client.connected()) {
67     reconnectMQTT();
68   }
69   client.loop();
70
71   // ส่งข้อมูลทุก 5 วินาที
72   if (millis() - lastMsg > 5000) {
73     lastMsg = millis();
74
75     // Data_11 : อุณหภูมิ 0.00 - 49.99
76     float temperature = random(0, 5000) / 100.0;
77
78     // Data_12 : ความชื้น 50.00 - 99.99
79     float humidity = random(5000, 10000) / 100.0;
80
81     // Data_13 : จำนวนผลิต Step Up
82     production++;
83
84     if (production > 999999) {
85       production = 0;
86     }
87
88     char payload[200];
89
90     snprintf(
91       payload,
92       sizeof(payload),
93       "{\"Data_21\":%.2f,\"Data_22\":%.2f,\"Data_23\":%lu",
94       temperature,
95       humidity,
96       production
97     );
98
99     Serial.print("Publish: ");
100    Serial.println(payload);
101    client.publish(mqtt_topic, payload);
102  }
103 }

```

Output Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on 'COM13')

ESP32 Code Text

```
#include <WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// ===== Wi-Fi =====
```

```
const char* ssid = "RUT956_T07";
```

```
const char* password = "12345667";
```

```
// ===== MQTT Broker =====
```

```
const char* mqtt_server = "192.168.1.165";
```

```
const int mqtt_port = 1883;
```

```
// Topic ของสถานี 1
```

```
const char* mqtt_topic = "factory/station2/data";

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMsg = 0;
unsigned long production = 0;

void setup_wifi() {
  delay(10);

  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi connected");
  Serial.print("ESP32 IP: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {

    Serial.print("Connecting to MQTT...");
```

```
String clientId = "ESP32C6-Station1-";
clientId += String(random(0xffff), HEX);

if (client.connect(clientId.c_str())) {
  Serial.println("connected");
} else {
  Serial.print("failed, rc=");
  Serial.print(client.state());
  Serial.println(" retry in 5 seconds");

  delay(5000);
}
}

void setup() {
  Serial.begin(115200);

  randomSeed(micros());

  setup_wifi();

  client.setServer(mqtt_server, mqtt_port);
}

void loop() {

  if (!client.connected()) {
    reconnectMQTT();
  }

  client.loop();
}
```

```

// ส่งข้อมูลทุก 5 วินาที
if (millis() - lastMsg > 5000) {
    lastMsg = millis();

    // Data_11 : อุณหภูมิ 0.00 - 49.99
    float temperature = random(0, 5000) / 100.0;

    // Data_12 : ความชื้น 50.00 - 99.99
    float humidity = random(5000, 10000) / 100.0;

    // Data_13 : จำนวนผลิต Step Up
    production++;

    if (production > 999999) {
        production = 0;
    }

    char payload[200];

    snprintf(
        payload,
        sizeof(payload),
        "{\"Data_21\":%.2f,\"Data_22\":%.2f,\"Data_23\":%lu}\",
        temperature,
        humidity,
        production
    );

    Serial.print("Publish: ");
    Serial.println(payload);

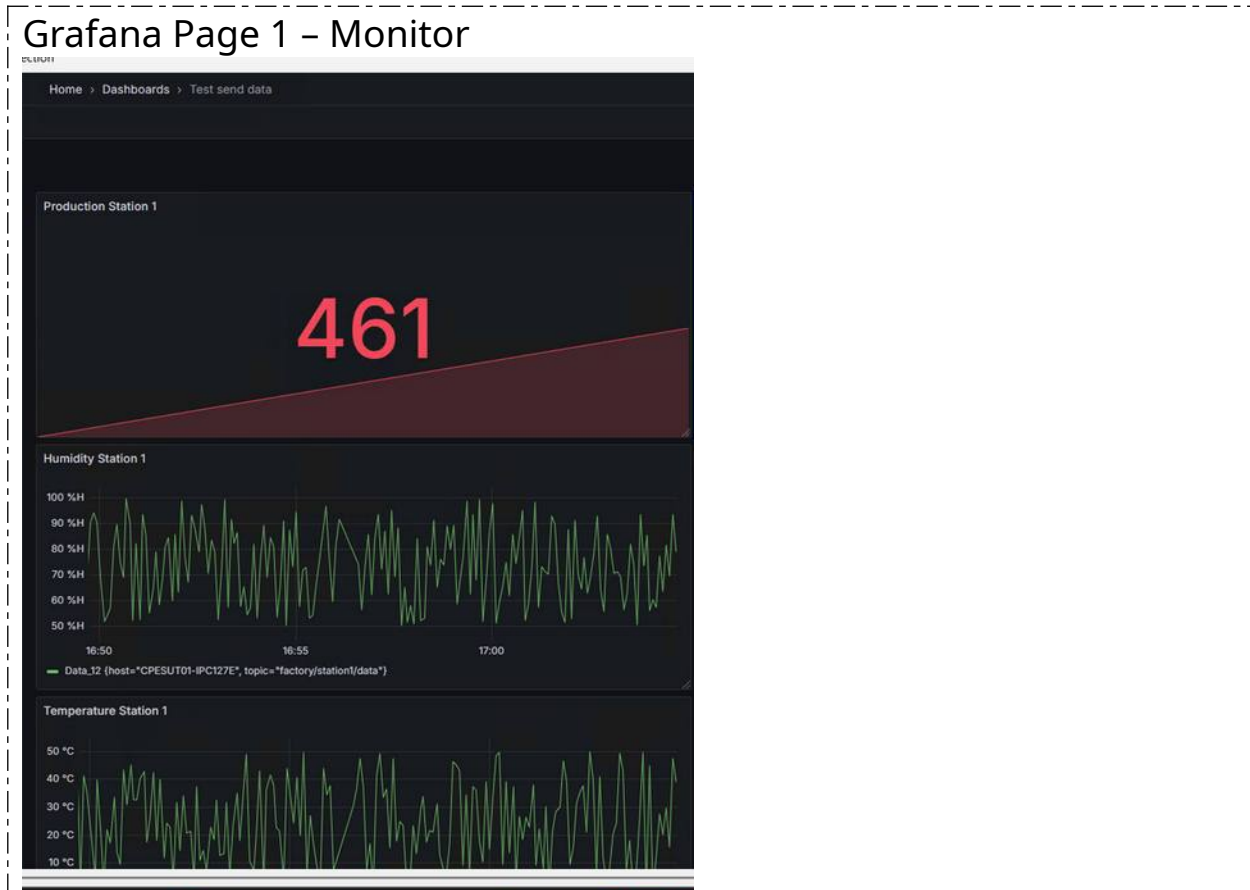
    client.publish(mqtt_topic, payload);
}

```

}

3. แสดงค่า Grafana Dashboard

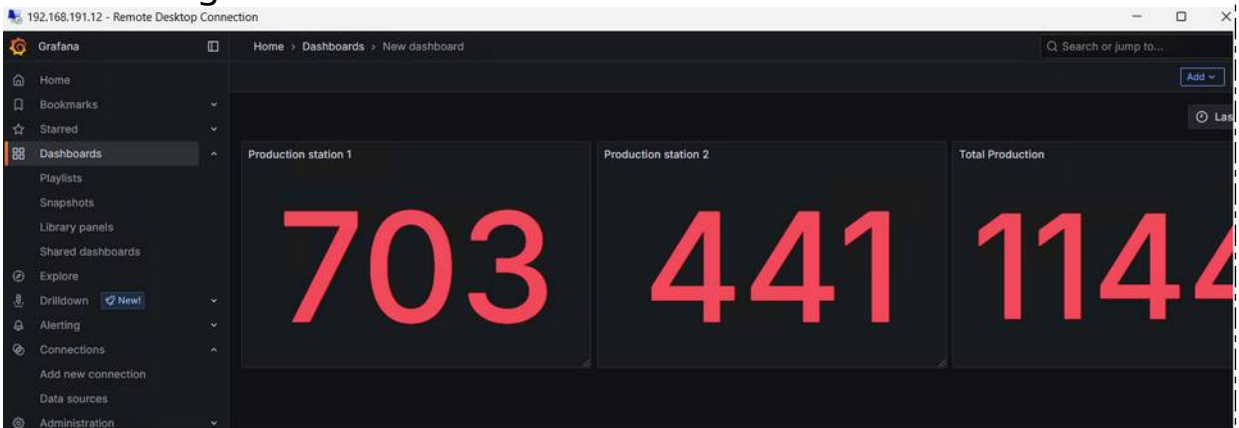
- เลือกแสดงค่าได้ 3 หน้า
- หน้า 1,2 แสดงค่า รายละเอียดของแต่ละสถานีผลิต
- หน้า 3 แสดงค่า จำนวนที่ผลิตแต่ละสถานีและจำนวนรวมที่ผลิตได้



Grafana Page 2 – Monitor



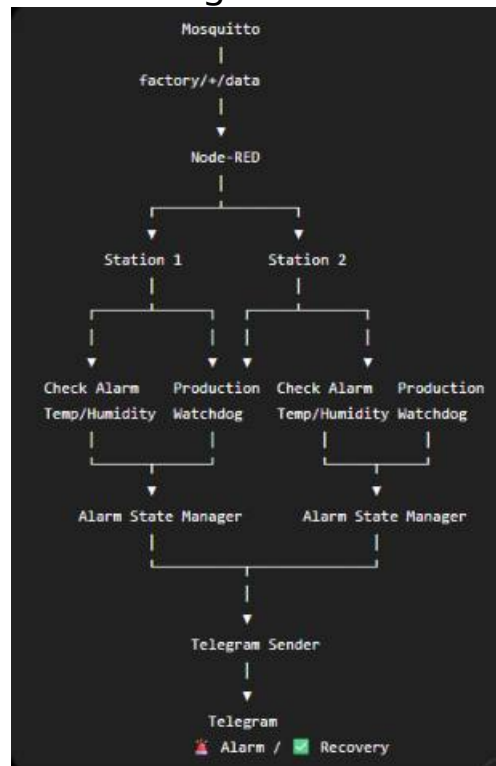
Grafana Page 3 – Monitor



Mission 6/7: Alarm if xxx

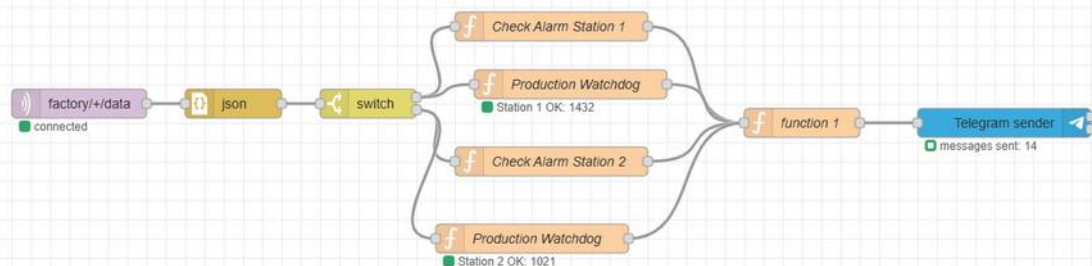
4. ภาพผังการทำงานโดยรวม

Overall Diagram



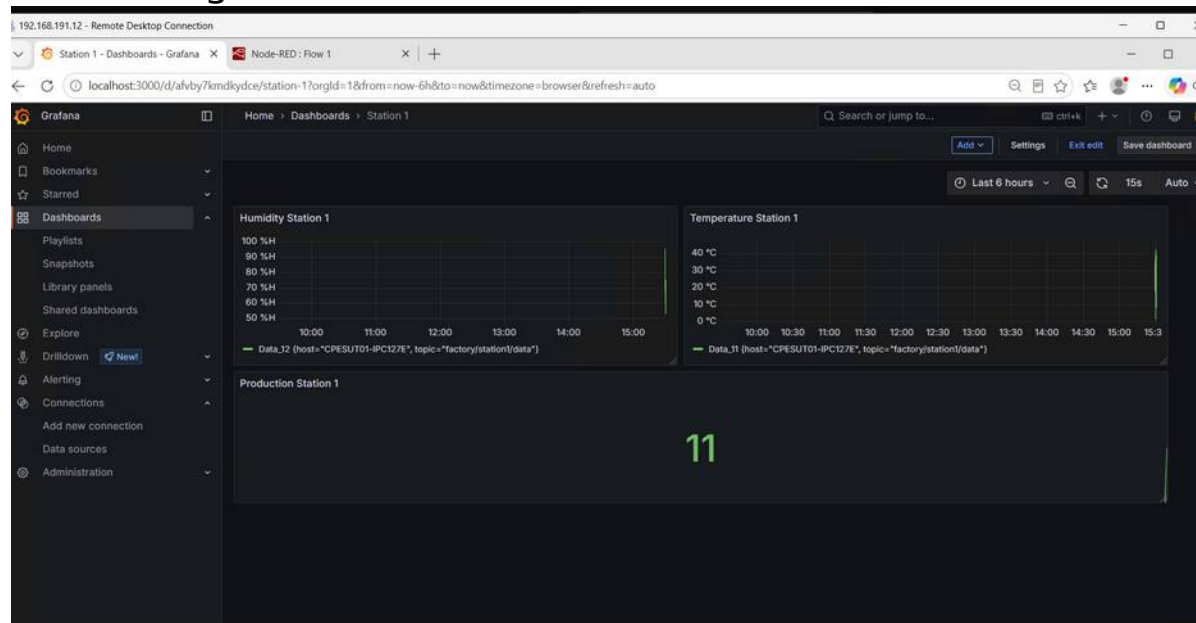
5. ใช้ NodeRED ร่วมกับ Telegram เพื่อสร้างการแจ้งเตือน

NodeRED Flow

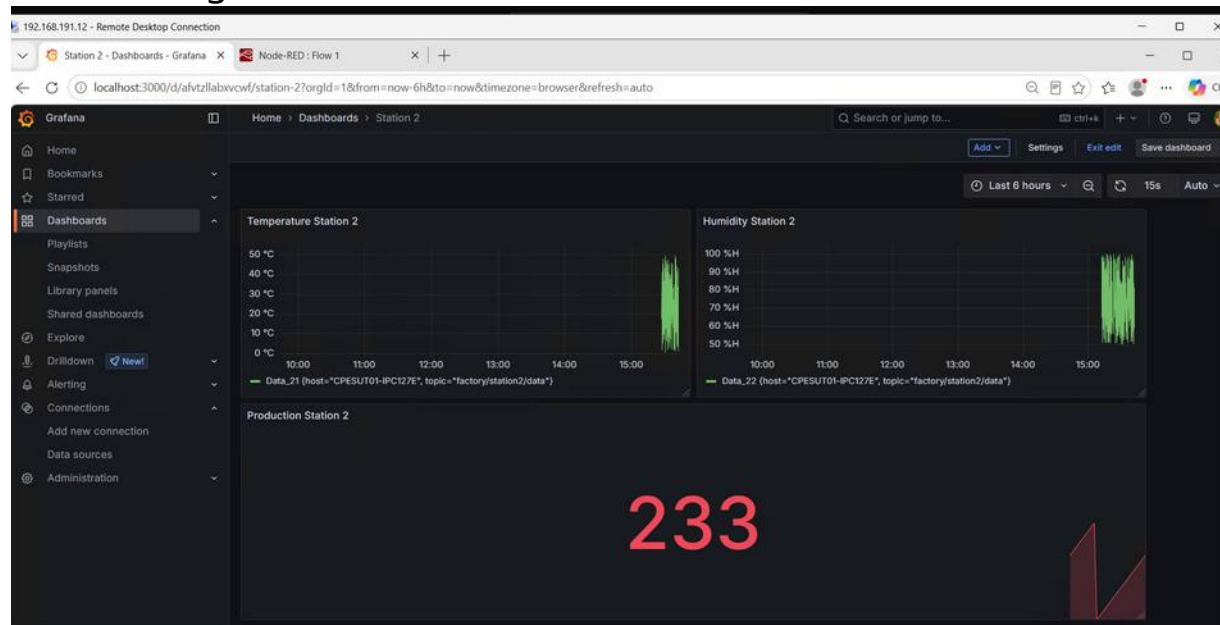


6. ปรับให้มี Alarm ไปยังผู้รับผิดชอบ กรณี อุณหภูมิผิดไปจากที่กำหนด (อาจ fix ค่า หรือตั้งผ่าน Dashboard)
7. ปรับให้มี Alarm ไปยังผู้รับผิดชอบ กรณี ความชื้นผิดไปจากที่กำหนด (อาจ fix ค่า หรือตั้งผ่าน Dashboard)
8. ปรับให้มี Alarm ไปยังผู้รับผิดชอบ กรณี จำนวนชิ้นงานที่ผลิตไม่เพิ่มขึ้น ในระยะเวลาที่กำหนด (อาจ fix ค่า หรือตั้งผ่าน Dashboard)

Grafana Page 1 – Monitor and Alarm



Grafana Page 2 - Monitor and Alarm



TEMPERATURE ALARM

Station: 2

Temperature: 49.25 °C

Allowed: 10 - 40 °C

HUMIDITY ALARM

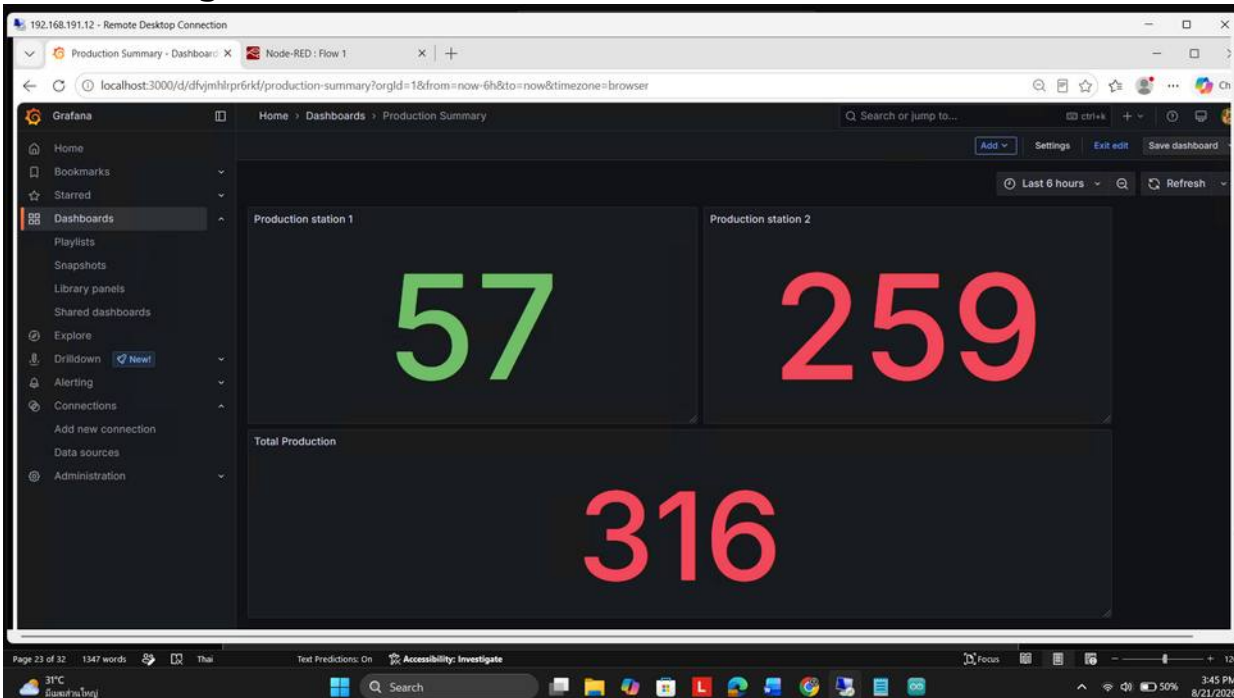
Station: 2

Humidity: 98.43 %

Allowed: 55 - 90 %

17:02

Grafana Page 3 - Monitor and Alarm



 **Production Alarm**
Station 2
Production has not increased for 30 seconds.
Current production: 1
15:22

 **TEMPERATURE RECOVERED**
Station: 2
Temperature: 27.09 °C

 **HUMIDITY ALARM**
Station: 2
Humidity: 99.81 %
Allowed: 55 - 90 %
15:22

Mission 7/7: Control from Dashboard

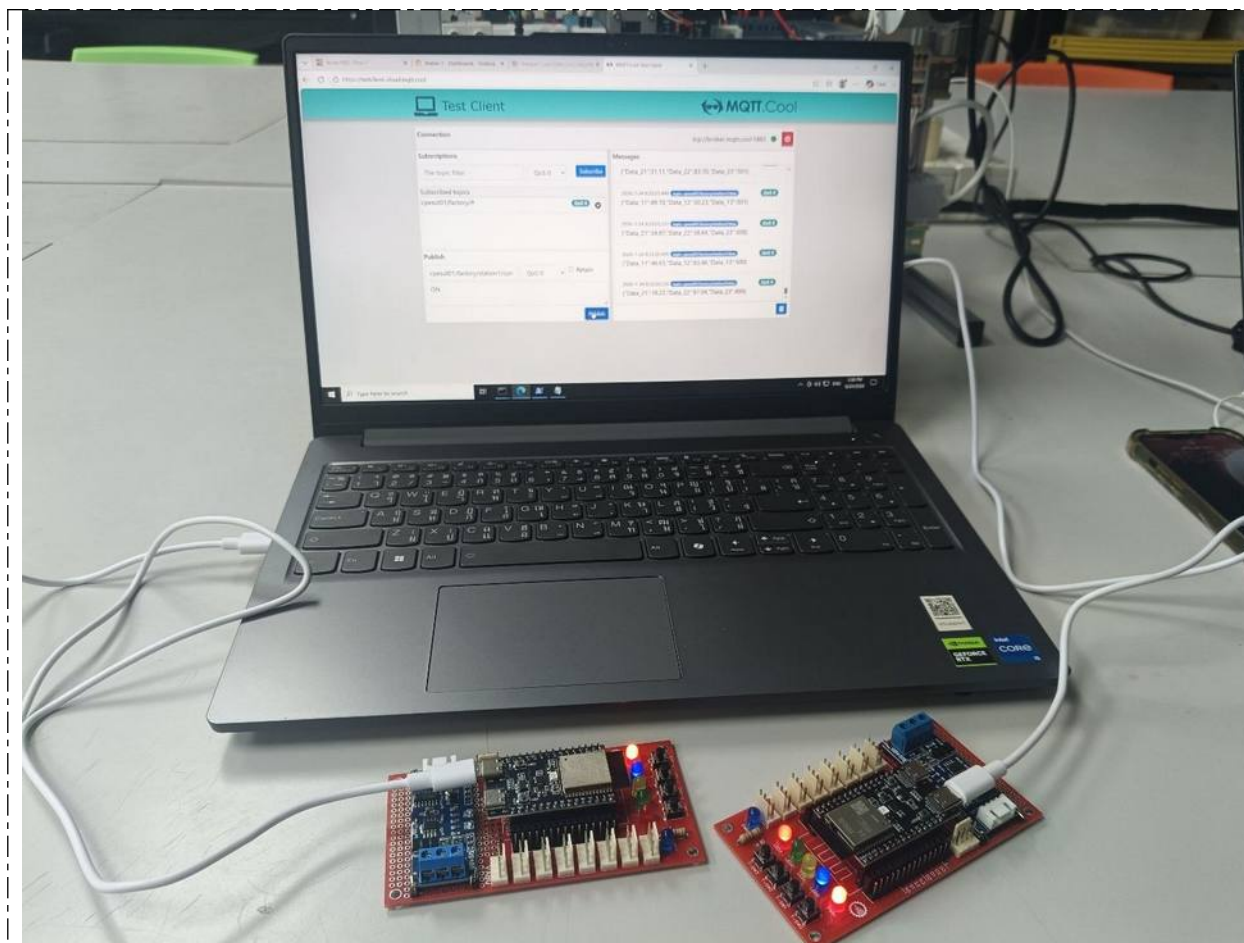
9. ภาพผังการทำงานโดยรวม

Overall Diagram

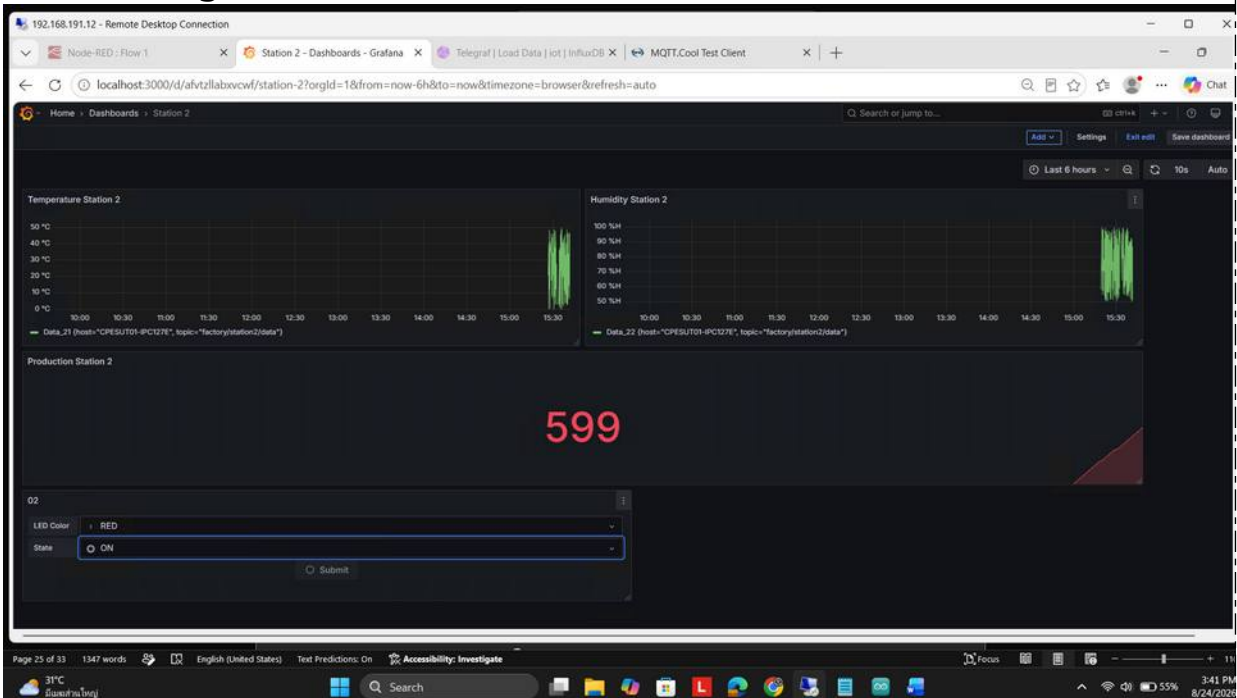


10. ต่อย่างจรให้มี 3 LED {RED, YLW, GRN} ที่สถานีทั้ง สอง
11. จาก Grafana Dashboard ให้สามารถควบคุม LED ได้ ผ่าน MQTT Broker (ให้ใช้ MQTT Cool : <https://testclient-cloud.mqtt.cool/> ในการควบคุมและมอนิเตอร์การทำงาน)

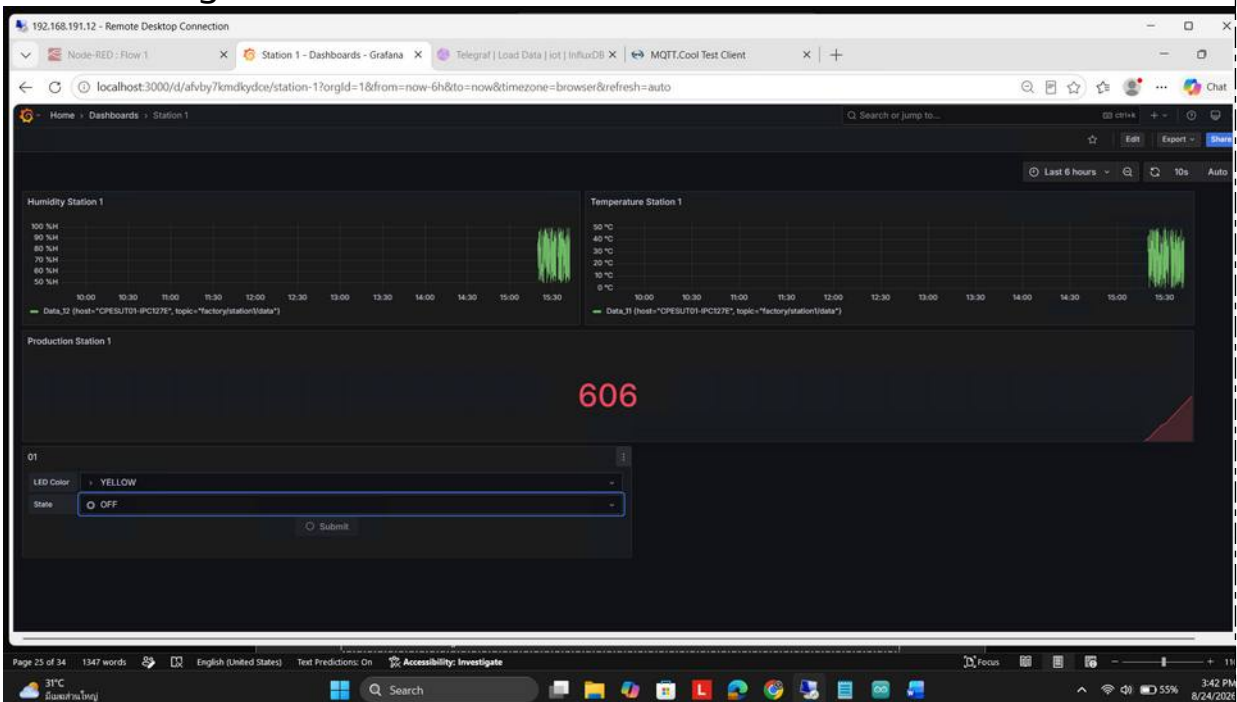
MQTT Cool for Monitor and Control



Grafana Page 1 – Monitor, Alarm and Control



Grafana Page 2 – Monitor, Alarm and Control



12. หากเกิด Alarm ตาม Mission2 ให้ LED ทั้ง 3 ตัวกระพริบ

ภาพถ่ายวงจร ESP32 ขณะทดสอบ



ESP32 Code Picture

Embedded_Week06 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

ESP32C6 Dev Module

Embedded_Week06.ino

```

60 Serial.println("ALARM MODE ON");
61
62 } else if (message == "OFF") {
63
64 }
65 alarmMode = false;
66
67 digitalWrite(LED_RED, LOW);
68 digitalWrite(LED_YELLOW, LOW);
69 digitalWrite(LED_GREEN, LOW);
70
71 Serial.println("ALARM MODE OFF");
72
73 }
74
75 return;
76
77 }
78
79 // ถ้า Alarm ตรวจจับพบ
80 // ไฟฟ้าจะ LED ปล่อยแสงออกมา
81 if (alarmMode) {
82   return;
83 }
84
85 // =====
86 // RED LED
87 // =====
88 if (String(topic) == "factory/station1/control/red") {
89
90   if (message == "ON") {
91     digitalWrite(LED_RED, HIGH);
92   } else {
93     digitalWrite(LED_RED, LOW);
94   }
95 }
96
97
98 // =====
99 // YELLOW LED
100 // =====
101 if (String(topic) == "factory/station1/control/yellow") {
102
103   if (message == "ON") {
104     digitalWrite(LED_YELLOW, HIGH);
105   } else {
106     digitalWrite(LED_YELLOW, LOW);
107   }
108 }
109
110
111 // =====
112 // GREEN LED
113 // =====
114 if (String(topic) == "factory/station1/control/green") {
115
116   if (message == "ON") {

```

Output Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on 'COM17')

Message: ON
ALARM MODE ON
Publish: ["Data_21":1.06,"Data_22":81.51,"Data_23":679]

Embedded_Week06 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

ESP32C6 Dev Module

Embedded_Week06.ino

```

141 Serial.print(".");
142
143 }
144
145 Serial.println();
146 Serial.println("WiFi connected");
147
148 Serial.print("ESP32 IP: ");
149 Serial.println(WiFi.localIP());
150
151 }
152
153 // =====
154 // MQTT RECONNECT
155 // =====
156 void reconnectMQTT() {
157
158   while (!client.connected()) {
159
160     Serial.print("Connecting to MQTT...");
161
162     String clientId = "ESP32C6-Station2-";
163     clientId += String(random(0xffff), HEX);
164
165     // กำหนดค่า
166     if (client.connect(clientId.c_str())) {
167
168       Serial.println("connected");
169
170       // กำหนดค่า Subscribe
171       client.subscribe("factory/station1/control/red");
172       client.subscribe("factory/station1/control/yellow");
173       client.subscribe("factory/station1/control/green");
174       client.subscribe("factory/station1/control/alarm");
175
176       Serial.println("Subscribed to control topics");
177
178     } else {
179
180       Serial.print("Failed, rc=");
181       Serial.print(client.state());
182
183       Serial.println(" retry in 5 seconds");
184
185       delay(5000);
186
187     }
188   }
189 }
190
191 // =====
192 // SETUP
193 // =====
194 void setup() {

```

Output Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on 'COM17')

Publish: ["Data_21":12.62,"Data_22":52.01,"Data_23":681]
Publish: ["Data_21":42.27,"Data_22":98.14,"Data_23":682]
Publish: ["Data_21":1.57,"Data_22":59.52,"Data_23":683]

ESP32 Code Text

```

#include <WiFi.h>
#include <PubSubClient.h>

#define LED_RED    12
#define LED_YELLOW 11
#define LED_GREEN  10

// ===== Wi-Fi =====
const char* ssid = "RUT956_T07";
const char* password = "12345667";

// ===== MQTT Broker =====
const char* mqtt_server = "192.168.1.165";
const int mqtt_port = 1883;

// ===== MQTT Topic =====
const char* mqtt_data_topic = "factory/station1/data";

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMsg = 0;
unsigned long production = 0;

// ===== Alarm Blink =====
bool alarmMode = false;
bool blinkState = false;
unsigned long lastBlink = 0;
const unsigned long blinkInterval = 500;

// =====
// MQTT CALLBACK
// =====

```

```
void callback(char* topic, byte* payload, unsigned int length) {

    String message = "";

    for (unsigned int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    message.trim();

    Serial.print("Topic: ");
    Serial.println(topic);

    Serial.print("Message: ");
    Serial.println(message);

    // =====
    // ALARM CONTROL
    // =====
    if (String(topic) == "factory/station1/control/alarm") {

        if (message == "ON") {

            alarmMode = true;

            Serial.println("ALARM MODE ON");

        } else if (message == "OFF") {

            alarmMode = false;

            digitalWrite(LED_RED, LOW);
            digitalWrite(LED_YELLOW, LOW);
```

```

    digitalWrite(LED_GREEN, LOW);

    Serial.println("ALARM MODE OFF");
}

return;
}

// ถ้า Alarm กำลังทำงาน
// ไม่ให้คำสั่ง LED ปกติมารบกวน
if (alarmMode) {
    return;
}

// =====
// RED LED
// =====
if (String(topic) == "factory/station1/control/red") {

    if (message == "ON") {
        digitalWrite(LED_RED, HIGH);
    } else {
        digitalWrite(LED_RED, LOW);
    }
}

// =====
// YELLOW LED
// =====
if (String(topic) == "factory/station1/control/yellow") {

    if (message == "ON") {
        digitalWrite(LED_YELLOW, HIGH);
    }
}

```

```
} else {
    digitalWrite(LED_YELLOW, LOW);
}
}

// =====
// GREEN LED
// =====
if (String(topic) == "factory/station1/control/green") {

    if (message == "ON") {
        digitalWrite(LED_GREEN, HIGH);
    } else {
        digitalWrite(LED_GREEN, LOW);
    }
}
}

// =====
// WIFI
// =====
void setup_wifi() {

    delay(10);

    Serial.println();
    Serial.print("Connecting to WiFi: ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
```

```

    delay(500);
    Serial.print(".");
}

Serial.println();
Serial.println("WiFi connected");

Serial.print("ESP32 IP: ");
Serial.println(WiFi.localIP());
}

// =====
// MQTT RECONNECT
// =====
void reconnectMQTT() {

    while (!client.connected()) {

        Serial.print("Connecting to MQTT...");

        String clientId = "ESP32C6-Station2-";
        clientId += String(random(0xffff), HEX);

        // เชื่อมก่อน
        if (client.connect(clientId.c_str())) {

            Serial.println("connected");

            // แล้วค่อย Subscribe
            client.subscribe("factory/station1/control/red");
            client.subscribe("factory/station1/control/yellow");
            client.subscribe("factory/station1/control/green");
            client.subscribe("factory/station1/control/alarm");

```

```
    Serial.println("Subscribed to control topics");

} else {

    Serial.print("failed, rc=");
    Serial.print(client.state());

    Serial.println(" retry in 5 seconds");

    delay(5000);
}
}
}

// =====
// SETUP
// =====
void setup() {

    Serial.begin(115200);

    randomSeed(micros());

    // LED
    pinMode(LED_RED, OUTPUT);
    pinMode(LED_YELLOW, OUTPUT);
    pinMode(LED_GREEN, OUTPUT);

    digitalWrite(LED_RED, LOW);
    digitalWrite(LED_YELLOW, LOW);
    digitalWrite(LED_GREEN, LOW);
```

```
setup_wifi();

client.setServer(mqtt_server, mqtt_port);

client.setCallback(callback);
}

// =====
// LOOP
// =====
void loop() {

  if (!client.connected()) {
    reconnectMQTT();
  }

  client.loop();

  // =====
  // ALARM BLINK
  // =====
  if (alarmMode) {

    if (millis() - lastBlink >= blinkInterval) {

      lastBlink = millis();

      blinkState = !blinkState;

      digitalWrite(LED_RED, blinkState);
      digitalWrite(LED_YELLOW, blinkState);
      digitalWrite(LED_GREEN, blinkState);
    }
  }
}
```

```

}

// =====
// SEND SENSOR DATA EVERY 5 SECONDS
// =====
if (millis() - lastMsg >= 5000) {

    lastMsg = millis();

    // Data_11 : Temperature 0.00 - 49.99
    float temperature =
        random(0, 5000) / 100.0;

    // Data_12 : Humidity 50.00 - 99.99
    float humidity =
        random(5000, 10000) / 100.0;

    // Data_13 : Production Step Up
    production++;

    if (production > 999999) {
        production = 0;
    }

    char payload[200];

    snprintf(
        payload,
        sizeof(payload),
        "{\"Data_11\":%.2f,\"Data_12\":%.2f,\"Data_13\":%lu}\",
        temperature,
        humidity,
        production

```

```
);  
  
Serial.print("Publish: ");  
Serial.println(payload);  
  
client.publish(  
    mqtt_data_topic,  
    payload  
);  
}  
}
```

IPC127 Embedded Industrial PC and Its Role as an IoT Gateway

1.Run On PC

1. Install Python Lib

- MediaPipe แนะนำให้ทำงานกับ Python 3.12.10
- Upgrade Pip >> python.exe -m pip install --upgrade pip
- pip install opencv-python==4.11.0.86
- pip install numpy==1.26.4
- pip install mediapipe==0.10.21
- pip install requests

2. Test Code_Version

```

1 import sys, cv2
2 import mediapipe
3 import numpy
4
5 print('Python = ',sys.version)
6 print('OpenCV = ',cv2.__version__)
7 print('Numpy = ',numpy.__version__)
8 print('MediaPipe = ',mediapipe.__version__)
9

```

```

Python = 3.12.9 | packaged by Anaconda, Inc. | (main, Feb 6 2025, 18:49:16) [MSC v.1929 64 bit (AMD64)]
OpenCV = 4.11.0
Numpy = 1.26.4
MediaPipe = 0.10.21

```

```

import sys, cv2
import mediapipe
import numpy

print('Python = ',sys.version)
print('OpenCV = ',cv2.__version__)
print('Numpy = ',numpy.__version__)
print('MediaPipe = ',mediapipe.__version__)

```


3. Test Code_OpenCAM

```

1
2 import cv2
3 cap = cv2.VideoCapture(0)
4 if not cap.isOpened():
5     print("ไม่สามารถเปิดกล้องได้")
6     exit()
7
8 while True:
9     ret, frame = cap.read()
10    if ret:
11        cv2.imshow('Camera Capture', frame)
12        if cv2.waitKey(1) & 0xFF == ord('q'):
13            break
14    else:
15        print("ไม่สามารถอ่านภาพจากกล้องได้")
16        break
17
18 cap.release()
19 cv2.destroyAllWindows()
20
21

```



```

import cv2
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("ไม่สามารถเปิดกล้องได้")
    exit()

while True:
    ret, frame = cap.read()
    if ret:
        cv2.imshow('Camera Capture', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    else:
        print("ไม่สามารถอ่านภาพจากกล้องได้")
        break

cap.release()
cv2.destroyAllWindows()

```

4. Test Code_MediaPipe

```

1 import cv2
2 import mediapipe as mp
3
4 # Initialize MediaPipe Hands
5 mp_hands = mp.solutions.hands
6 hands = mp_hands.Hands()
7 mp_drawing = mp.solutions.drawing_utils
8
9 # Function to detect the state of each finger
10 def count_fingers(hand_landmarks):
11     # Detect finger states (up or down)
12     thumb_up = hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_TIP].x < hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_BASE].x
13     index_up = hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_BASE].y
14     middle_up = hand_landmarks.landmark[mp_hands.HandLandmark.MIDDLE_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.MIDDLE_FINGER_BASE].y
15     ring_up = hand_landmarks.landmark[mp_hands.HandLandmark.RING_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.RING_FINGER_BASE].y
16     pinky_up = hand_landmarks.landmark[mp_hands.HandLandmark.PINKY_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.PINKY_BASE].y
17     # Combine finger statuses into a list
18     finger_status = [thumb_up, index_up, middle_up, ring_up, pinky_up]
19     return finger_status
20
21 # Initialize VideoCapture
22 cap = cv2.VideoCapture(0)
23 while cap.isOpened():
24     ret, frame = cap.read()
25     if not ret:
26         break
27     frame = cv2.flip(frame, 1)
28     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
29     # Detect hand landmarks
30     results = hands.process(frame_rgb)
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
34             fingers = count_fingers(hand_landmarks)
35             cv2.putText(frame, f"Fingers: {fingers}", (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0))
36     cv2.imshow('Hand Gesture Recognition', frame)
37     if cv2.waitKey(5) & 0xFF == 27: # Exit on pressing 'Esc'
38         break
39 cap.release()
40 cv2.destroyAllWindows()
41

```



```

import cv2
import mediapipe as mp

# Initialize MediaPipe Hands
mp_hands = mp.solutions.hands
hands = mp_hands.Hands()
mp_drawing = mp.solutions.drawing_utils

# Function to detect the state of each finger
def count_fingers(hand_landmarks):
    # Detect finger states (up or down)
    thumb_up = hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_TIP].x < hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_PIP].x
    index_up = hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_PIP].y
    middle_up = hand_landmarks.landmark[mp_hands.HandLandmark.MIDDLE_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.MIDDLE_FINGER_PIP].y
    ring_up = hand_landmarks.landmark[mp_hands.HandLandmark.RING_FINGER_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.RING_FINGER_PIP].y
    pinky_up = hand_landmarks.landmark[mp_hands.HandLandmark.PINKY_TIP].y < hand_landmarks.landmark[mp_hands.HandLandmark.PINKY_PIP].y
    # Combine finger statuses into a list
    finger_status = [thumb_up, index_up, middle_up, ring_up, pinky_up]
    return finger_status

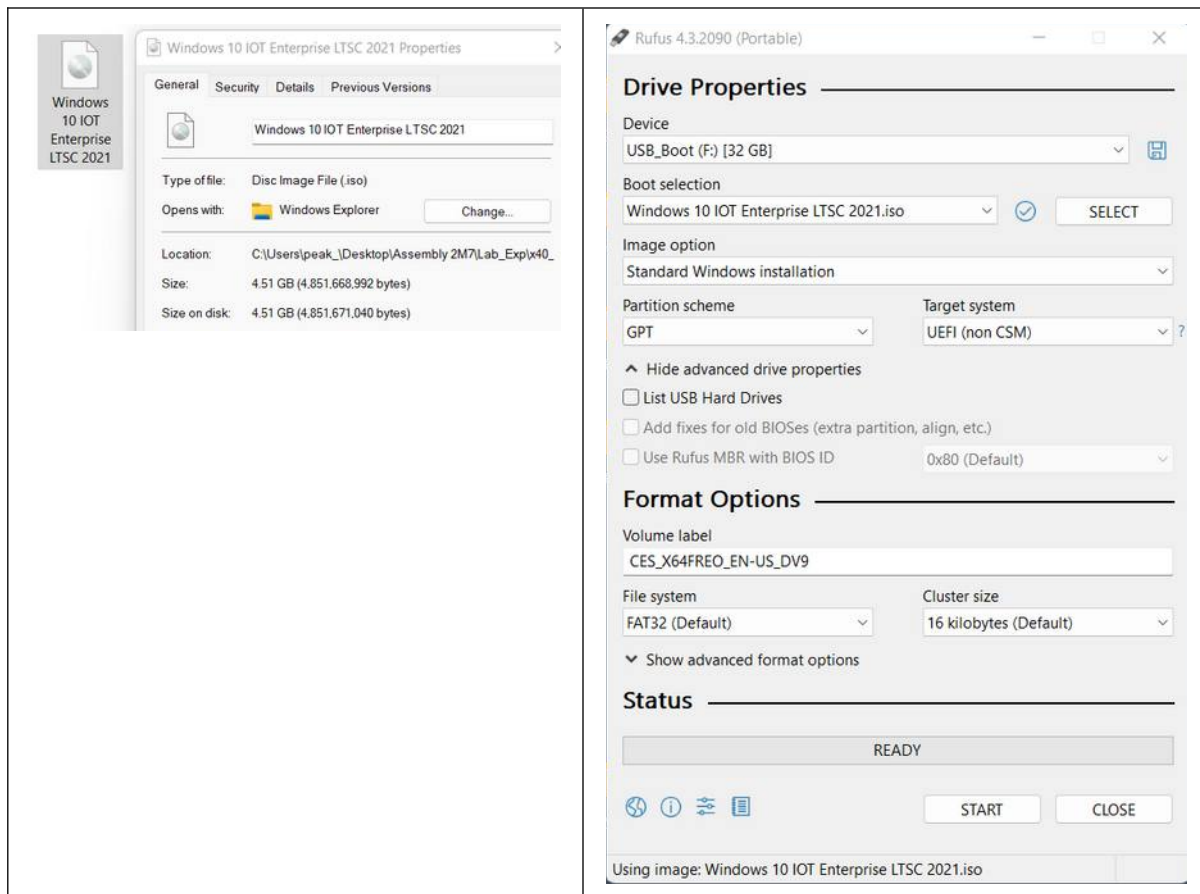
# Initialize VideoCapture
cap = cv2.VideoCapture(0)
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    # Detect hand landmarks
    results = hands.process(frame_rgb)
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
            fingers = count_fingers(hand_landmarks)
            cv2.putText(frame, f'Fingers: {fingers}', (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
    cv2.imshow('Hand Gesture Recognition', frame)
    if cv2.waitKey(5) & 0xFF == 27: # Exit on pressing 'Esc'
        break
cap.release()
cv2.destroyAllWindows()

```

2. Install OS on IPC127

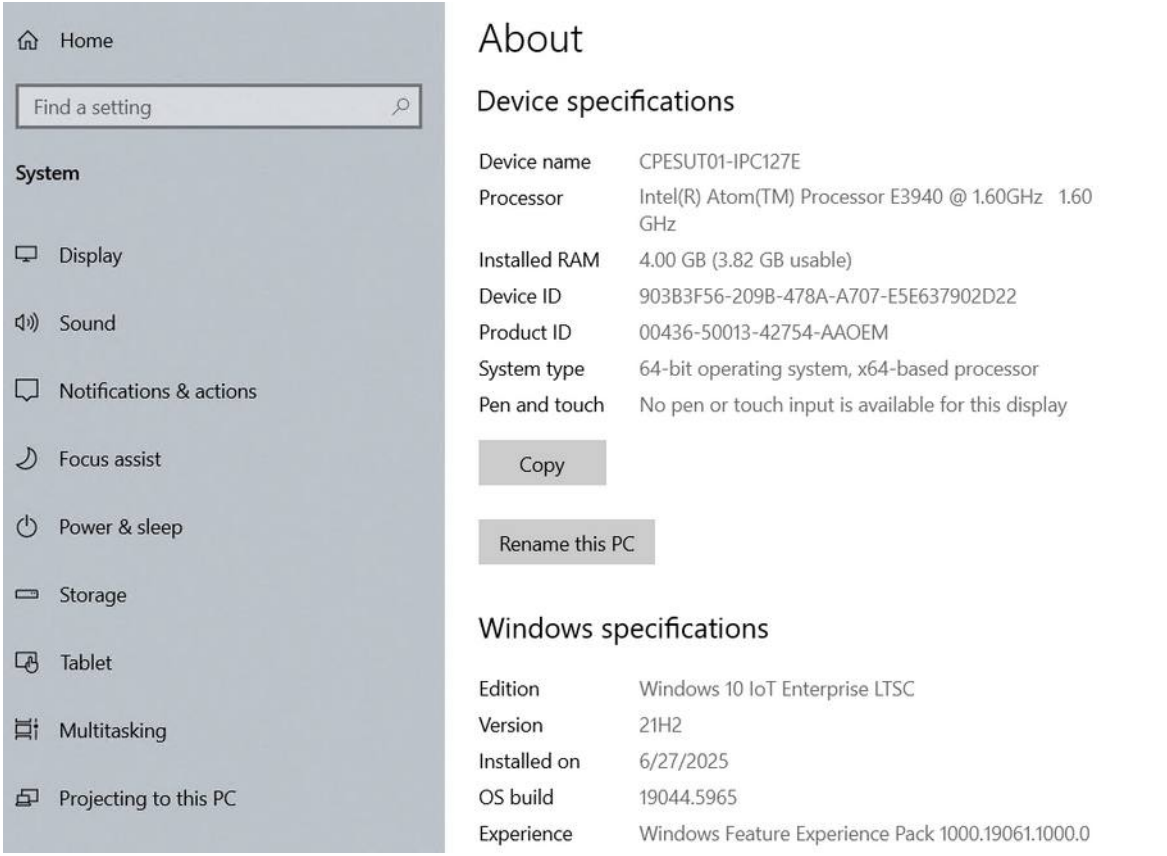
Install Win-10 IOT OS

1. Download “Windows 10 IOT Enterprise LTSC 2021.iso”
2. สร้าง USB Boot Drive ด้วยโปรแกรม “rufus-4.3p.exe”



3. IPC127 เข้า Boot Manu ด้วยการกด ESC Key

4. เลือก USB Boot และติดตั้ง Win10 IOT



The screenshot shows the Windows Settings application, specifically the 'About' page. The left sidebar lists various settings categories: Home, Find a setting, System, Display, Sound, Notifications & actions, Focus assist, Power & sleep, Storage, Tablet, Multitasking, and Projecting to this PC. The main content area is titled 'About' and is divided into two sections: 'Device specifications' and 'Windows specifications'.

About

Device specifications

Device name	CPESUT01-IPC127E
Processor	Intel(R) Atom(TM) Processor E3940 @ 1.60GHz 1.60 GHz
Installed RAM	4.00 GB (3.82 GB usable)
Device ID	903B3F56-209B-478A-A707-E5E637902D22
Product ID	00436-50013-42754-AAOEM
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

Buttons: Copy, Rename this PC

Windows specifications

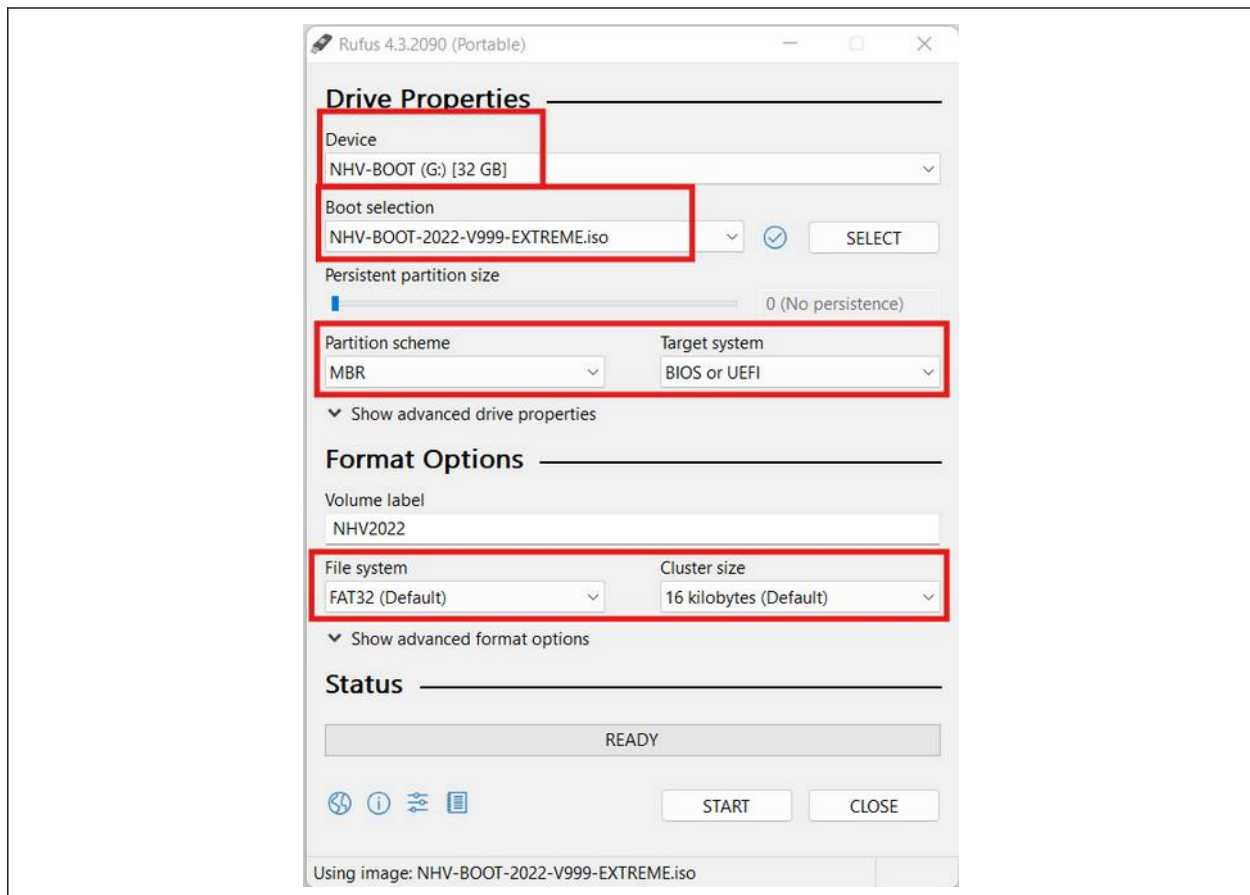
Edition	Windows 10 IoT Enterprise LTSC
Version	21H2
Installed on	6/27/2025
OS build	19044.5965
Experience	Windows Feature Experience Pack 1000.19061.1000.0

เพื่อความสะดวก กำหนดให้ตั้งค่า login=**cpe28sut**,
password=**adminZ01**

Clone OS

1. เรียกโปรแกรม rufus-4.3p.exe

- เขียนไฟล์ USBBOOT “NHV-BOOT-2022-V999-EXTREME.iso” ไปยัง USB Drive



2. BOOT SIEMENS IPC127 จาก USB_Drive <IPC127 เข้า BOOT Manu ด้วยการกด ESC Key>

3. เรียกใช้โปรแกรม Symantec Ghost

- กรณีเก็บ HDD : Ghost >> Disk to Image
- กรณีเขียน HDD : Ghost >> Image to Disk

3. เปิดใช้งาน Remote Desktop

4. ติดตั้ง Python 3.12.10 และทดสอบการทำงาน

- MediaPipe แนะนำให้ทำงานกับ Python 3.12.10
- Upgrade Pip >> python.exe -m pip install --upgrade pip

5. Add Library < opencv-python, mediapipe, requests >

- pip install opencv-python==4.11.0.86
- pip install numpy==1.26.4
- pip install mediapipe==0.10.21
- pip install requests

6. Install OS ทดสอบ mediapipe

7. ทดสอบ mediapipe ส่งค่าไปแสดงที่ 5 LED

Add Arduino Lib >>

<https://github.com/me-no-dev/ESPAsyncWebServer/archive/refs/heads/master.zip>

8. ทดสอบ mediapipe ส่งค่าไปแสดงที่ 10 LED

9. ปรับแต่งให้ โปรแกรมทำงานทันทีเมื่อเปิดเครื่อง IPC217